

Representation Learning With PCA and Autoencoders

Group Number: 123

#	Student Name	Email
1	Agarwal Rankit Atulkumar	2025ab05296@wilp.bits-pilani.ac.in
2	Nishant Verma	2025aa05051@wilp.bits-pilani.ac.in
3	Aryasomayajula Subrahmanya Srinivas	2025aa05539@wilp.bits-pilani.ac.in
4	Wali Haider	2025ab05295@wilp.bits-pilani.ac.in
5	Amit Mallikarjun Masuti	2024ac05742@wilp.bits-pilani.ac.in

This notebook solves the assignment tasks for MNIST and CIFAR10 converted to gray level.

Requirements implemented:

- MNIST and CIFAR10 loaded from `tensorflow.keras.datasets`
- CIFAR10 converted to gray level
- both datasets resized to 28 x 28
- intensities rescaled to [50, 200]
- random 70% / 20% / 10% train / validation / test split
- standard PCA and randomized PCA with 30 components
- logistic regression classification on PCA features and autoencoder features
- multiclass ROC curves
- reconstruction SNR in dB
- tied-weight linear autoencoder with unit-magnitude encoder vectors
- component/weight visualization and quantitative PCA-AE comparison
- deep convolutional autoencoder with 30-dimensional latent space
- single hidden layer and three hidden layer dense autoencoder comparisons

Before running:

```
# Uncomment if your environment is missing packages.  
# %pip install -q tensorflow scikit-learn matplotlib pandas seaborn
```

```
In [1]: import os
import socket
import random
from datetime import datetime
import warnings
from dataclasses import dataclass

# Keep Matplotlib cache inside a writable location when running from managed
# kernels.
os.environ.setdefault("MPLCONFIGDIR", os.path.join("/tmp", "matplotlib"))
os.makedirs(os.environ["MPLCONFIGDIR"], exist_ok=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

import sklearn
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    roc_curve,
    auc,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize, StandardScaler

from scipy.optimize import linear_sum_assignment

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
```

Configuration

`FAST_MODE=False` runs the full 70/20/10 split required by the assignment. If you only want to verify that the notebook executes, set `FAST_MODE=True`.

```

In [2]: RANDOM_SEED = 42
FAST_MODE = False

N_COMPONENTS = 30
IMAGE_SHAPE = (28, 28)
N_CLASSES = 10

BATCH_SIZE = 256
LINEAR_AE_EPOCHS = 150
DENSE_AE_EPOCHS = 60
CONV_AE_EPOCHS = 50
PATIENCE = 12

if FAST_MODE:
    # Small values for quick testing only. Leave FAST_MODE=False for submission
    runs.
    MAX_TOTAL_PER_DATASET = 12000
    LINEAR_AE_EPOCHS = 10
    DENSE_AE_EPOCHS = 8
    CONV_AE_EPOCHS = 8
else:
    MAX_TOTAL_PER_DATASET = None

np.random.seed(RANDOM_SEED)
random.seed(RANDOM_SEED)
tf.random.set_seed(RANDOM_SEED)

print("TensorFlow:", tf.__version__)
print("scikit-learn:", sklearn.__version__)
print("FAST_MODE:", FAST_MODE)
gpus = tf.config.list_physical_devices("GPU")
print("GPU devices:", gpus)
for gpu in gpus:
    details = tf.config.experimental.get_device_details(gpu)
    print("GPU name:", details.get("device_name", "unknown"))
print("Hostname:", socket.gethostname())
print("Date/Time:", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))

```

```

TensorFlow: 2.20.0
scikit-learn: 1.6.1
FAST_MODE: False
GPU devices: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
GPU name: Tesla T4
Hostname: 9f350ba3d94f
Date/Time: 2026-06-08 12:18:08

```

Helper Functions

```

In [3]: @dataclass
class DatasetSplit:
    name: str
    x_train_img_50_200: np.ndarray
    y_train: np.ndarray
    x_val_img_50_200: np.ndarray
    y_val: np.ndarray
    x_test_img_50_200: np.ndarray
    y_test: np.ndarray

    @property
    def x_train_flat_50_200(self):
        return self.x_train_img_50_200.reshape(len(self.x_train_img_50_200), -1)

    @property
    def x_val_flat_50_200(self):
        return self.x_val_img_50_200.reshape(len(self.x_val_img_50_200), -1)

    @property
    def x_test_flat_50_200(self):
        return self.x_test_img_50_200.reshape(len(self.x_test_img_50_200), -1)

    @property
    def x_train_01(self):
        return (self.x_train_img_50_200 - 50.0) / 150.0

    @property
    def x_val_01(self):
        return (self.x_val_img_50_200 - 50.0) / 150.0

    @property
    def x_test_01(self):
        return (self.x_test_img_50_200 - 50.0) / 150.0

    @property
    def x_train_flat_01(self):
        return self.x_train_01.reshape(len(self.x_train_01), -1)

    @property
    def x_val_flat_01(self):
        return self.x_val_01.reshape(len(self.x_val_01), -1)

    @property
    def x_test_flat_01(self):
        return self.x_test_01.reshape(len(self.x_test_01), -1)

def set_all_seeds(seed=RANDOM_SEED):
    np.random.seed(seed)
    random.seed(seed)
    tf.random.set_seed(seed)

def to_50_200(x):
    """Map image intensities from [0, 255] to [50, 200]."""
    x = x.astype("float32")
    return 50.0 + (x / 255.0) * 150.0

def preprocess_mnist_images(x):
    x = x.astype("float32")
    if x.ndim == 3:
        x = x[..., np.newaxis]
    # MNIST is already 28 x 28; keep this call for a consistent pipeline.
    x = tf.image.resize(x, IMAGE_SHAPE, method="bilinear").numpy()
    return to_50_200(x)

```

```

def preprocess_cifar10_images(x):
    x = x.astype("float32")
    # ITU-R BT.601 luminance conversion.
    gray = 0.299 * x[..., 0] + 0.587 * x[..., 1] + 0.114 * x[..., 2]
    gray = gray[..., np.newaxis]
    gray = tf.image.resize(gray, IMAGE_SHAPE, method="bilinear").numpy()
    return to_50_200(gray)

def make_split(name, x, y, max_total=None):
    y = y.reshape(-1).astype("int64")

    if max_total is not None and max_total < len(x):
        rng = np.random.default_rng(RANDOM_SEED)
        idx = rng.choice(len(x), size=max_total, replace=False)
        x = x[idx]
        y = y[idx]

    # First split off 10% test, then split the remaining 90% into 70/20.
    x_temp, x_test, y_temp, y_test = train_test_split(
        x,
        y,
        test_size=0.10,
        random_state=RANDOM_SEED,
        stratify=y,
    )
    val_fraction_of_temp = 0.20 / 0.90
    x_train, x_val, y_train, y_val = train_test_split(
        x_temp,
        y_temp,
        test_size=val_fraction_of_temp,
        random_state=RANDOM_SEED,
        stratify=y_temp,
    )

    return DatasetSplit(name, x_train, y_train, x_val, y_val, x_test, y_test)

def mean_snr_db(original, reconstructed, eps=1e-10):
    """Average per-image SNR in dB: 10 log10(signal_power / noise_power)."""
    original = original.reshape(len(original), -1).astype("float64")
    reconstructed = reconstructed.reshape(len(reconstructed), -1).astype("float64")
    signal_power = np.mean(original**2, axis=1)
    noise_power = np.mean((original - reconstructed) ** 2, axis=1)
    return float(np.mean(10.0 * np.log10((signal_power + eps) / (noise_power +
eps))))

def plot_component_grid(components, title, n_rows=5, n_cols=6,
image_shape=IMAGE_SHAPE, cmap="gray"):
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(10, 8))
    fig.suptitle(title, fontsize=15, y=0.98)
    for i, ax in enumerate(axes.ravel()):
        comp = components[i].reshape(image_shape)
        ax.imshow(comp, cmap=cmap)
        ax.set_title(f"{i + 1}", fontsize=8)
        ax.axis("off")
    plt.tight_layout()
    plt.show()

def plot_reconstruction_examples(original, reconstructed, title, n=8):
    original = original[:n].reshape(n, *IMAGE_SHAPE)
    reconstructed = reconstructed[:n].reshape(n, *IMAGE_SHAPE)
    fig, axes = plt.subplots(2, n, figsize=(1.7 * n, 3.6))

```

```

fig.suptitle(title, fontsize=14)
for i in range(n):
    axes[0, i].imshow(original[i], cmap="gray", vmin=50, vmax=200)
    axes[0, i].axis("off")
    axes[0, i].set_title("Test", fontsize=8)
    axes[1, i].imshow(reconstructed[i], cmap="gray", vmin=50, vmax=200)
    axes[1, i].axis("off")
    axes[1, i].set_title("Recon", fontsize=8)
plt.tight_layout()
plt.show()

def fit_logistic_and_roc(z_train, y_train, z_test, y_test, title):
    clf = LogisticRegression(
        max_iter=1500,
        solver="lbfgs",
        n_jobs=-1,
        random_state=RANDOM_SEED,
    )
    clf.fit(z_train, y_train)
    y_pred = clf.predict(z_test)
    y_score = clf.predict_proba(z_test)

    acc = accuracy_score(y_test, y_pred)
    macro_auc = roc_auc_score(y_test, y_score, multi_class="ovr", average="macro")

    y_test_bin = label_binarize(y_test, classes=np.arange(N_CLASSES))
    fpr = {}
    tpr = {}
    roc_auc = {}
    for class_id in range(N_CLASSES):
        fpr[class_id], tpr[class_id], _ = roc_curve(y_test_bin[:, class_id],
y_score[:, class_id])
        roc_auc[class_id] = auc(fpr[class_id], tpr[class_id])

    plt.figure(figsize=(8, 6))
    for class_id in range(N_CLASSES):
        plt.plot(fpr[class_id], tpr[class_id], lw=1.2, label=f"Class {class_id}")
    AUC={roc_auc[class_id]:.3f}")
    plt.plot([0, 1], [0, 1], "k--", lw=1)
    plt.xlabel("False positive rate")
    plt.ylabel("True positive rate")
    plt.title(f"{title}\nAccuracy={acc:.4f}, macro AUC={macro_auc:.4f}")
    plt.legend(fontsize=8, ncol=2)
    plt.tight_layout()
    plt.show()

    return clf, {"accuracy": acc, "macro_auc": macro_auc}

def compare_component_spaces(pca_components, ae_components, dataset_name):
    """Compare PCA components and AE weight vectors.

    Both inputs should have shape (n_components, n_pixels).
    Sign is ignored because eigenvectors are sign-ambiguous.
    """
    pca_norm = pca_components / np.linalg.norm(pca_components, axis=1,
keepdims=True)
    ae_norm = ae_components / np.linalg.norm(ae_components, axis=1, keepdims=True)

    abs_cos = np.abs(pca_norm @ ae_norm.T)
    row_ind, col_ind = linear_sum_assignment(-abs_cos)
    matched = abs_cos[row_ind, col_ind]

    singular_values = np.linalg.svd(pca_norm @ ae_norm.T, compute_uv=False)
    principal_angles_deg = np.degrees(np.arccos(np.clip(singular_values, -1, 1)))

```

```

summary = {
    "dataset": dataset_name,
    "mean_matched_abs_cosine": float(np.mean(matched)),
    "median_matched_abs_cosine": float(np.median(matched)),
    "min_matched_abs_cosine": float(np.min(matched)),
    "mean_subspace_singular_value": float(np.mean(singular_values)),
    "max_principal_angle_deg": float(np.max(principal_angles_deg)),
}
return summary, abs_cos, matched, principal_angles_deg

def make_reconstruction_dataset(x, training=False):
    ds = tf.data.Dataset.from_tensor_slices(x.astype("float32"))
    if training:
        ds = ds.shuffle(min(len(x), 10000), seed=RANDOM_SEED,
reshuffle_each_iteration=True)
    return ds.batch(BATCH_SIZE).map(lambda batch: (batch, batch),
num_parallel_calls=tf.data.AUTOTUNE).prefetch(tf.data.AUTOTUNE)

def make_input_dataset(x):
    return
tf.data.Dataset.from_tensor_slices(x.astype("float32")).batch(BATCH_SIZE).prefetch(
tf.data.AUTOTUNE)

def early_stopping():
    return keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=PATIENCE,
        restore_best_weights=True,
    )

def to_keras_images_01(x_01):
    return x_01.astype("float32")

def flat01_to_50_200(x_flat_01):
    x_flat_01 = np.clip(x_flat_01, 0.0, 1.0)
    return 50.0 + 150.0 * x_flat_01

```

Load and Preprocess Datasets

```
In [4]: print("Loading MNIST and CIFAR10 from Keras...")
(mnist_x_train_raw, mnist_y_train_raw), (mnist_x_test_raw, mnist_y_test_raw) =
keras.datasets.mnist.load_data()
(cifar_x_train_raw, cifar_y_train_raw), (cifar_x_test_raw, cifar_y_test_raw) =
keras.datasets.cifar10.load_data()

mnist_x_raw = np.concatenate([mnist_x_train_raw, mnist_x_test_raw], axis=0)
mnist_y_raw = np.concatenate([mnist_y_train_raw, mnist_y_test_raw], axis=0)
cifar_x_raw = np.concatenate([cifar_x_train_raw, cifar_x_test_raw], axis=0)
cifar_y_raw = np.concatenate([cifar_y_train_raw, cifar_y_test_raw], axis=0)

mnist_x = preprocess_mnist_images(mnist_x_raw)
cifar_x = preprocess_cifar10_images(cifar_x_raw)

mnist = make_split("MNIST", mnist_x, mnist_y_raw, max_total=MAX_TOTAL_PER_DATASET)
cifar10_gray = make_split("CIFAR10-gray", cifar_x, cifar_y_raw,
max_total=MAX_TOTAL_PER_DATASET)
datasets = [mnist, cifar10_gray]

for ds in datasets:
    print(
        f"{ds.name}: train={len(ds.y_train)}, validation={len(ds.y_val)}, test=
{len(ds.y_test)}, "
        f"range=({ds.x_train_img_50_200.min():.1f},
{ds.x_train_img_50_200.max():.1f})"
    )
```

Loading MNIST and CIFAR10 from Keras...

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 2s 0us/step

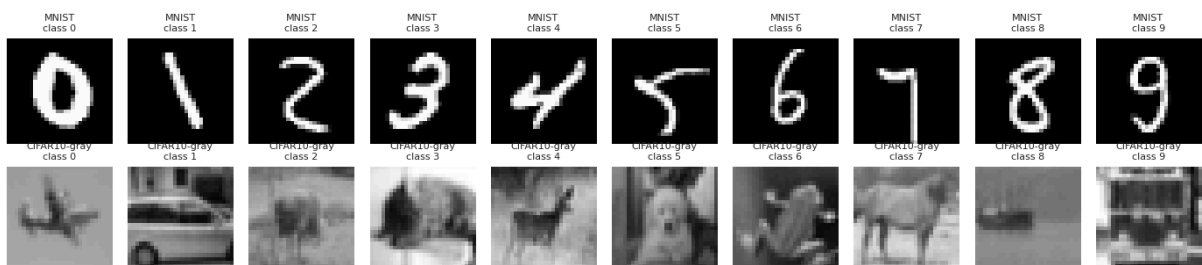
Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>

170498071/170498071 ————— 30s 0us/step

MNIST: train=48999, validation=14001, test=7000, range=(50.0, 200.0)

CIFAR10-gray: train=42000, validation=12000, test=6000, range=(50.0, 200.0)

```
In [5]: fig, axes = plt.subplots(2, 10, figsize=(14, 3.2))
for row, ds in enumerate(datasets):
    for col in range(10):
        idx = np.where(ds.y_train == col)[0][0]
        axes[row, col].imshow(ds.x_train_img_50_200[idx].squeeze(), cmap="gray",
vmin=50, vmax=200)
        axes[row, col].set_title(f"{ds.name}\n{col} class {col}", fontsize=8)
        axes[row, col].axis("off")
plt.tight_layout()
plt.show()
```



Task 1: Standard PCA, Randomized PCA, Logistic Regression, ROC and SNR

For each dataset, PCA is fitted using the 70% training split. The first 30 principal components are then used as features for a multinomial logistic regression classifier. Reconstruction SNR is computed on the 10% test split.

The top-30 eigenvalues are tabulated explicitly after PCA fitting, and the matching component vectors are displayed as 28 x 28 grayscale images.

```

In [6]: task1_rows = []
task1_models = {}

for ds in datasets:
    for solver_name, svd_solver in [("standard PCA", "full"), ("randomized PCA",
"randomized")]:
        print(f"\n{ds.name}: fitting {solver_name}")
        pca = PCA(
            n_components=N_COMPONENTS,
            svd_solver=svd_solver,
            random_state=RANDOM_SEED,
            iterated_power=5 if svd_solver == "randomized" else "auto",
        )
        pca.fit(ds.x_train_flat_50_200)

        z_train = pca.transform(ds.x_train_flat_50_200)
        z_test = pca.transform(ds.x_test_flat_50_200)
        recon_test = pca.inverse_transform(z_test)
        snr = mean_snr_db(ds.x_test_flat_50_200, recon_test)

        clf, metrics = fit_logistic_and_roc(
            z_train,
            ds.y_train,
            z_test,
            ds.y_test,
            title=f"{ds.name}: Logistic Regression on {solver_name} features",
        )

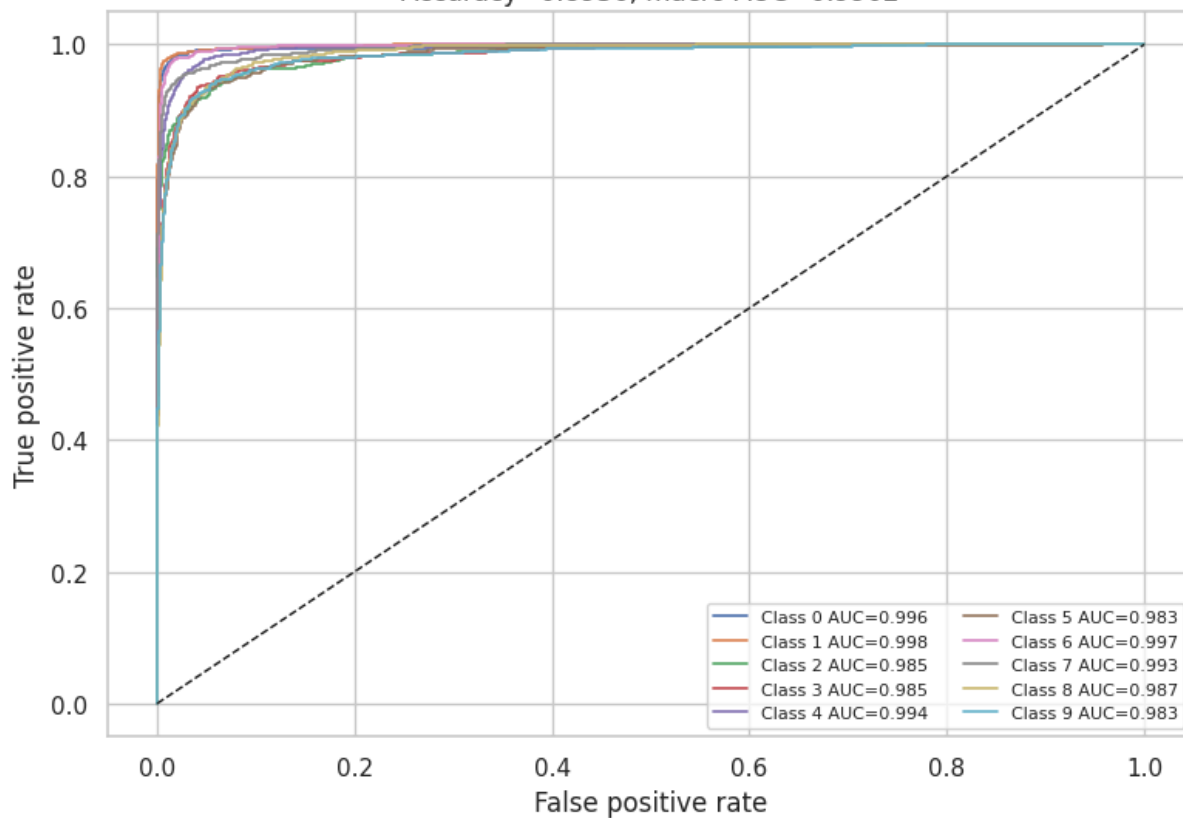
        task1_rows.append(
            {
                "dataset": ds.name,
                "method": solver_name,
                "components": N_COMPONENTS,
                "accuracy": metrics["accuracy"],
                "macro_auc_ovr": metrics["macro_auc"],
                "explained_variance_ratio_sum":
float(np.sum(pca.explained_variance_ratio_)),
                "average_test_snr_db": snr,
            }
        )
        task1_models[(ds.name, solver_name)] = {
            "pca": pca,
            "clf": clf,
            "z_train": z_train,
            "z_test": z_test,
            "recon_test": recon_test,
        }

task1_results = pd.DataFrame(task1_rows)
task1_results

```

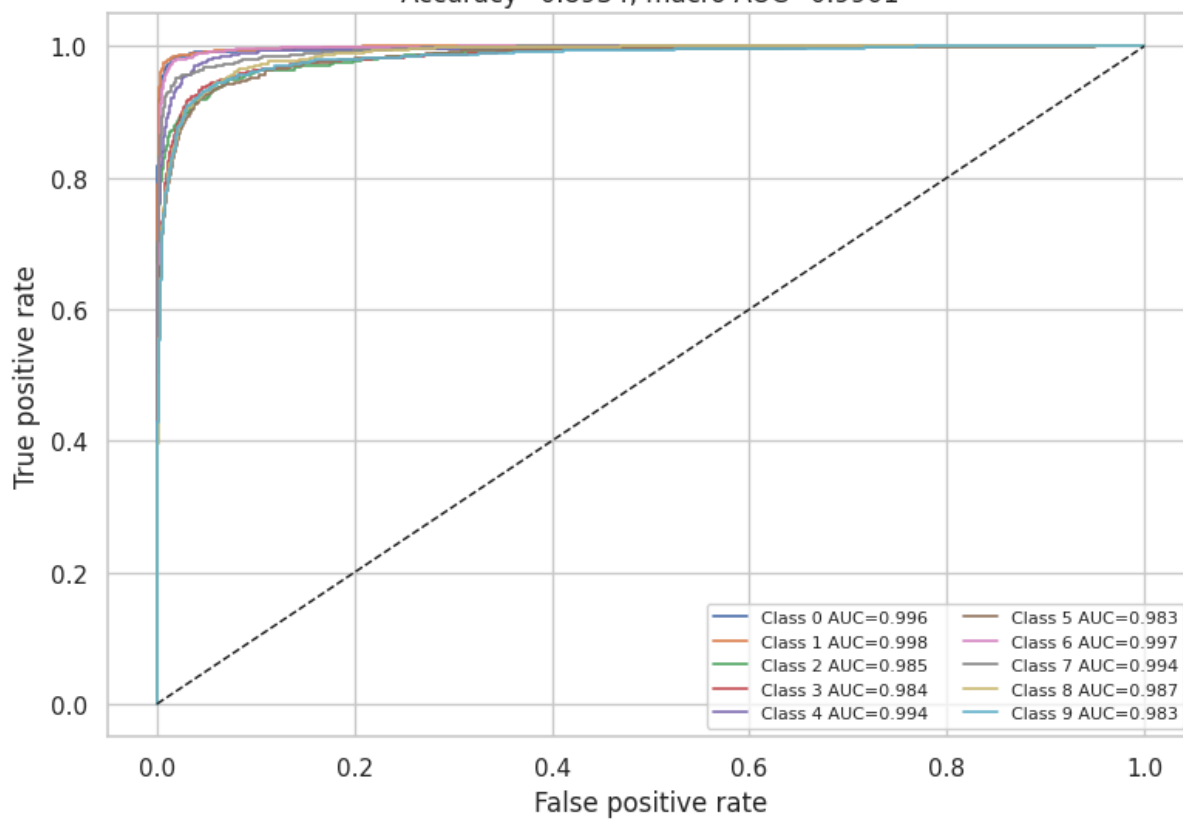
MNIST: fitting standard PCA

MNIST: Logistic Regression on standard PCA features
Accuracy=0.8936, macro AUC=0.9902



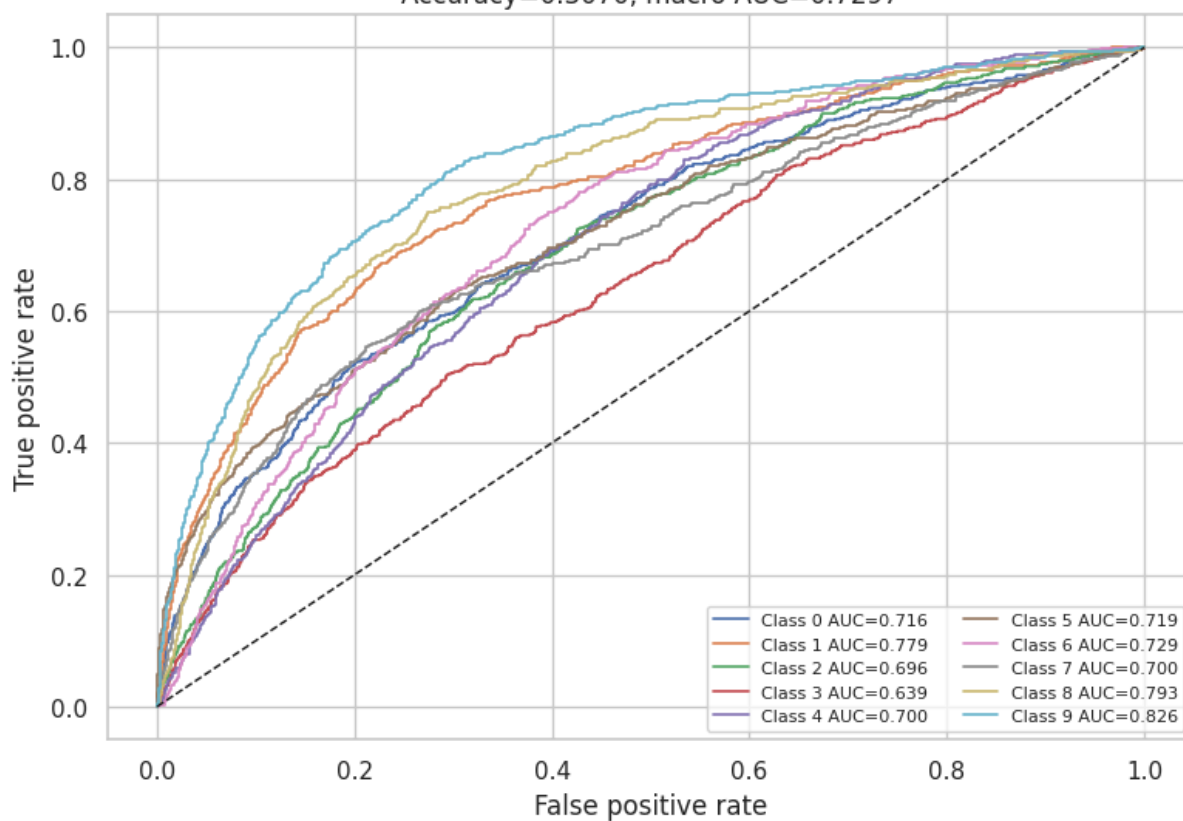
MNIST: fitting randomized PCA

MNIST: Logistic Regression on randomized PCA features
Accuracy=0.8934, macro AUC=0.9901



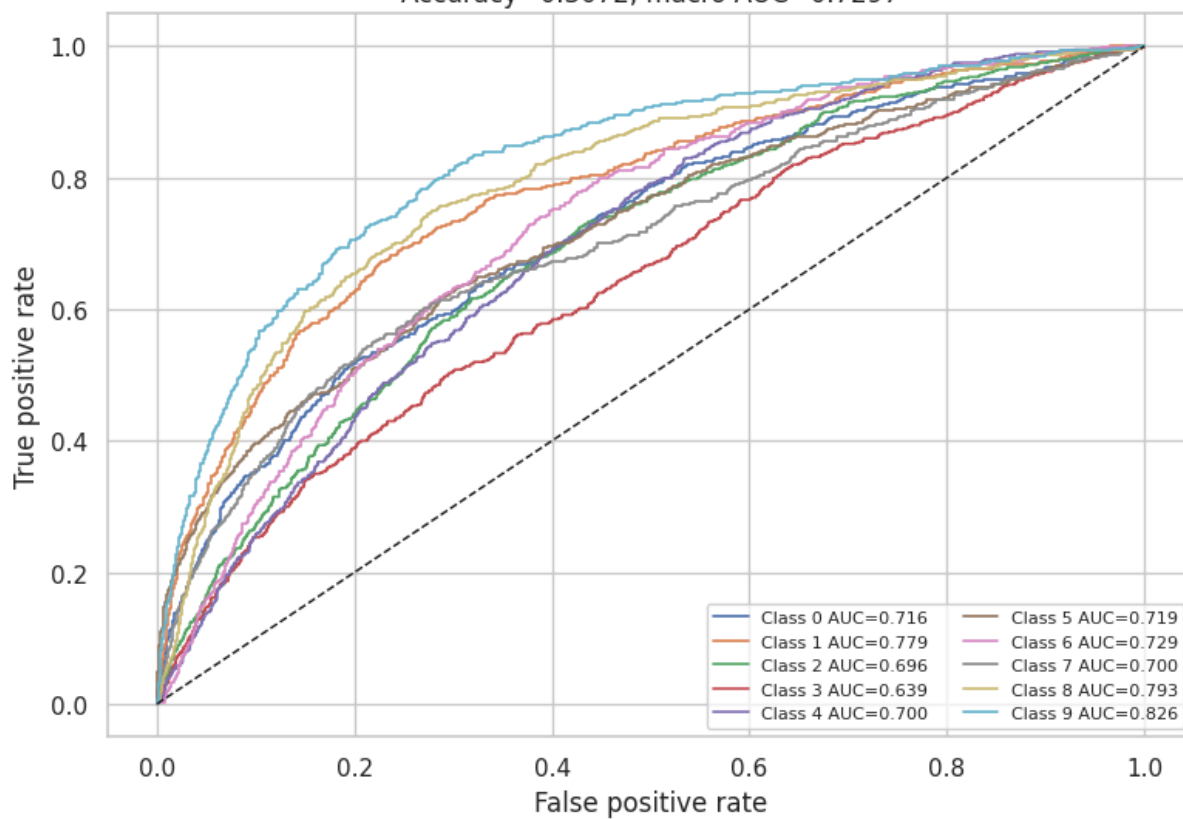
CIFAR10-gray: fitting standard PCA

CIFAR10-gray: Logistic Regression on standard PCA features
Accuracy=0.3070, macro AUC=0.7297



CIFAR10-gray: fitting randomized PCA

CIFAR10-gray: Logistic Regression on randomized PCA features
Accuracy=0.3072, macro AUC=0.7297



Out[6]:

	dataset	method	components	accuracy	macro_auc_ovr	explained_variance_ratio_sum	average_test_snr_db
0	MNIST	standard PCA	30	0.893571	0.990232	0.731537	12.572198
1	MNIST	randomized PCA	30	0.893429	0.990141	0.731483	12.571397
2	CIFAR10-gray	standard PCA	30	0.307000	0.729698	0.849658	20.001223
3	CIFAR10-gray	randomized PCA	30	0.307167	0.729697	0.849626	20.001208

```

In [7]: # Top 30 eigenvalues and associated principal component indices for each PCA
variant
# sklearn.PCA.explained_variance_ stores the eigenvalues of the covariance matrix.
eigenvalue_rows = []
for (ds_name, solver_name), info in task1_models.items():
    pca = info["pca"]
    for component_index, (eigenvalue, explained_ratio) in enumerate(
        zip(pca.explained_variance_, pca.explained_variance_ratio_),
        start=1,
    ):
        eigenvalue_rows.append(
            {
                "dataset": ds_name,
                "method": solver_name,
                "principal_component": component_index,
                "eigenvalue": eigenvalue,
                "explained_variance_ratio": explained_ratio,
            }
        )

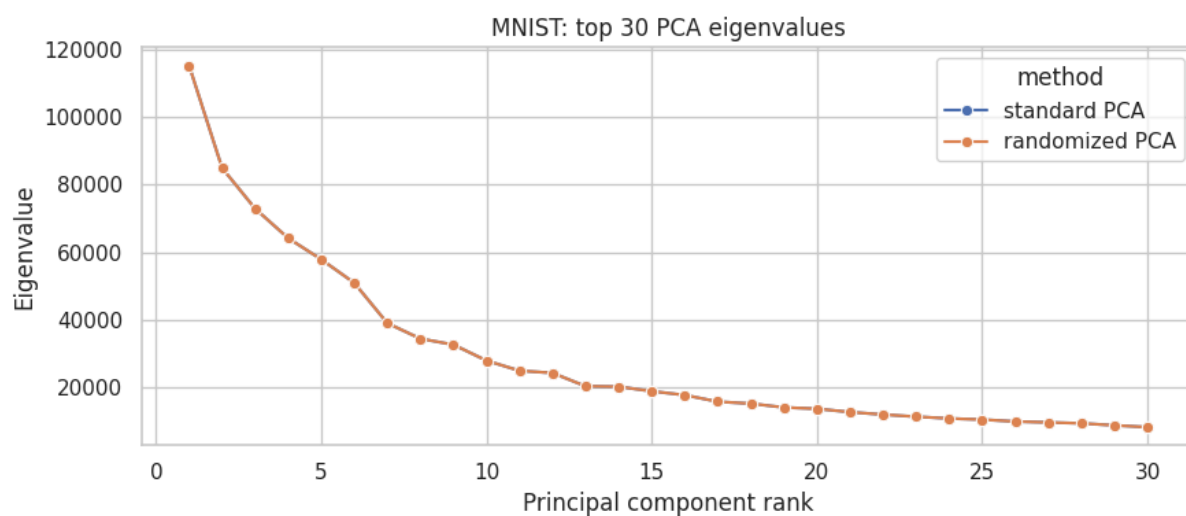
task1_top30_eigenvalues = pd.DataFrame(eigenvalue_rows)
display(task1_top30_eigenvalues)

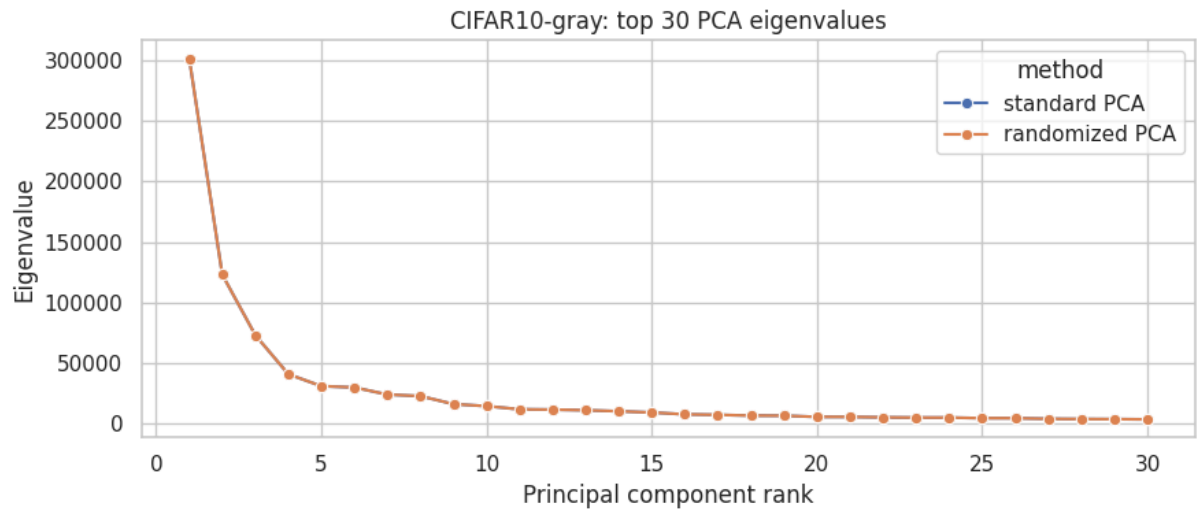
for ds_name in task1_top30_eigenvalues["dataset"].unique():
    plt.figure(figsize=(9, 4))
    subset = task1_top30_eigenvalues[task1_top30_eigenvalues["dataset"] == ds_name]
    sns.lineplot(
        data=subset,
        x="principal_component",
        y="eigenvalue",
        hue="method",
        marker="o",
    )
    plt.title(f"{ds_name}: top 30 PCA eigenvalues")
    plt.xlabel("Principal component rank")
    plt.ylabel("Eigenvalue")
    plt.tight_layout()
    plt.show()

```

	dataset	method	principal_component	eigenvalue	explained_variance_ratio
0	MNIST	standard PCA	1	115334.734375	0.097198
1	MNIST	standard PCA	2	84966.359375	0.071605
2	MNIST	standard PCA	3	72920.585938	0.061453
3	MNIST	standard PCA	4	64178.867188	0.054086
4	MNIST	standard PCA	5	57888.882812	0.048785
...	...	...	...	...	...
115	CIFAR10-gray	randomized PCA	26	4224.890625	0.004475
116	CIFAR10-gray	randomized PCA	27	3719.742920	0.003940
117	CIFAR10-gray	randomized PCA	28	3548.849121	0.003759
118	CIFAR10-gray	randomized PCA	29	3444.521240	0.003648
119	CIFAR10-gray	randomized PCA	30	3264.271973	0.003457

120 rows × 5 columns





```
In [8]: # Explicit comparison: standard PCA vs randomized PCA
for ds_name in task1_results['dataset'].unique():
    std = task1_results[(task1_results['dataset']==ds_name) &
(task1_results['method']=='standard PCA')].iloc[0]
    rnd = task1_results[(task1_results['dataset']==ds_name) &
(task1_results['method']=='randomized PCA')].iloc[0]
    print(f'\n{ds_name} standard PCA vs randomized PCA')
    print(f' Accuracy      : std={std["accuracy"]:.4f} rnd={rnd["accuracy"]:.4f}
delta={rnd["accuracy"]-std["accuracy"]:.4f}')
    print(f' Macro AUC (OvR): std={std["macro_auc_ovr"]:.4f} rnd=
{rnd["macro_auc_ovr"]:.4f} delta={rnd["macro_auc_ovr"]-
std["macro_auc_ovr"]:.4f}')
    print(f' Explained Var. : std={std["explained_variance_ratio_sum"]:.4f} rnd=
{rnd["explained_variance_ratio_sum"]:.4f} delta=
{rnd["explained_variance_ratio_sum"]-std["explained_variance_ratio_sum"]:.4f}')
    print(f' Test SNR (dB) : std={std["average_test_snr_db"]:.4f} rnd=
{rnd["average_test_snr_db"]:.4f} delta={rnd["average_test_snr_db"]-
std["average_test_snr_db"]:.4f}')
```

MNIST standard PCA vs randomized PCA

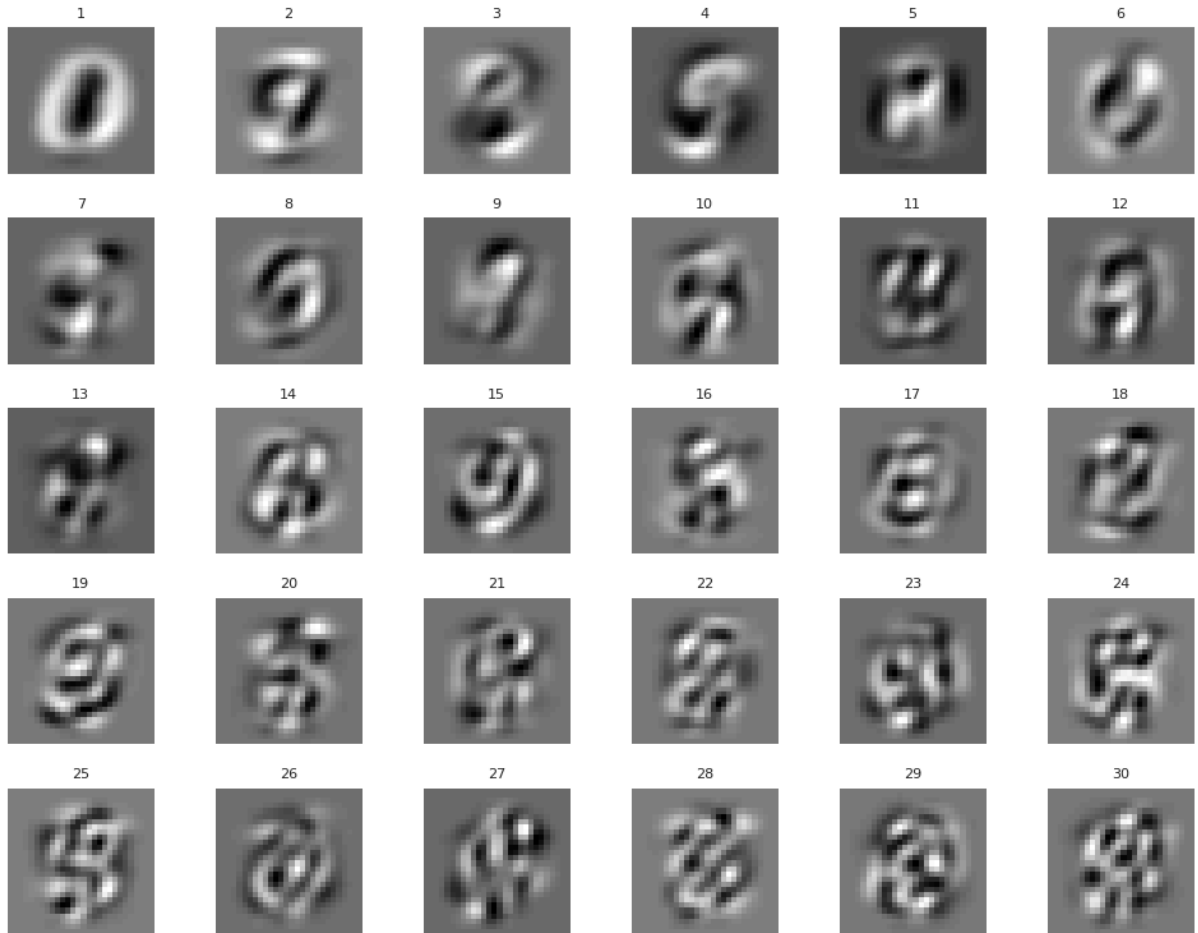
Accuracy	: std=0.8936	rnd=0.8934	delta=-0.0001
Macro AUC (OvR):	std=0.9902	rnd=0.9901	delta=-0.0001
Explained Var. :	std=0.7315	rnd=0.7315	delta=-0.0001
Test SNR (dB) :	std=12.5722	rnd=12.5714	delta=-0.0008

CIFAR10-gray standard PCA vs randomized PCA

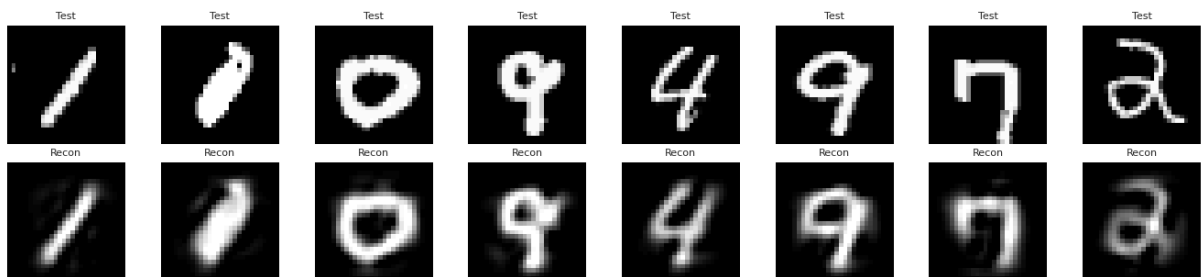
Accuracy	: std=0.3070	rnd=0.3072	delta=+0.0002
Macro AUC (OvR):	std=0.7297	rnd=0.7297	delta=-0.0000
Explained Var. :	std=0.8497	rnd=0.8496	delta=-0.0000
Test SNR (dB) :	std=20.0012	rnd=20.0012	delta=-0.0000


```
In [9]: for ds in datasets:
        pca = task1_models[(ds.name, "standard PCA")]["pca"]
        plot_component_grid(pca.components_, f"{ds.name}: top 30 standard PCA
        eigenvectors")
        recon = task1_models[(ds.name, "standard PCA")]["recon_test"]
        plot_reconstruction_examples(ds.x_test_flat_50_200, recon, f"{ds.name}:
        standard PCA reconstruction examples")
```

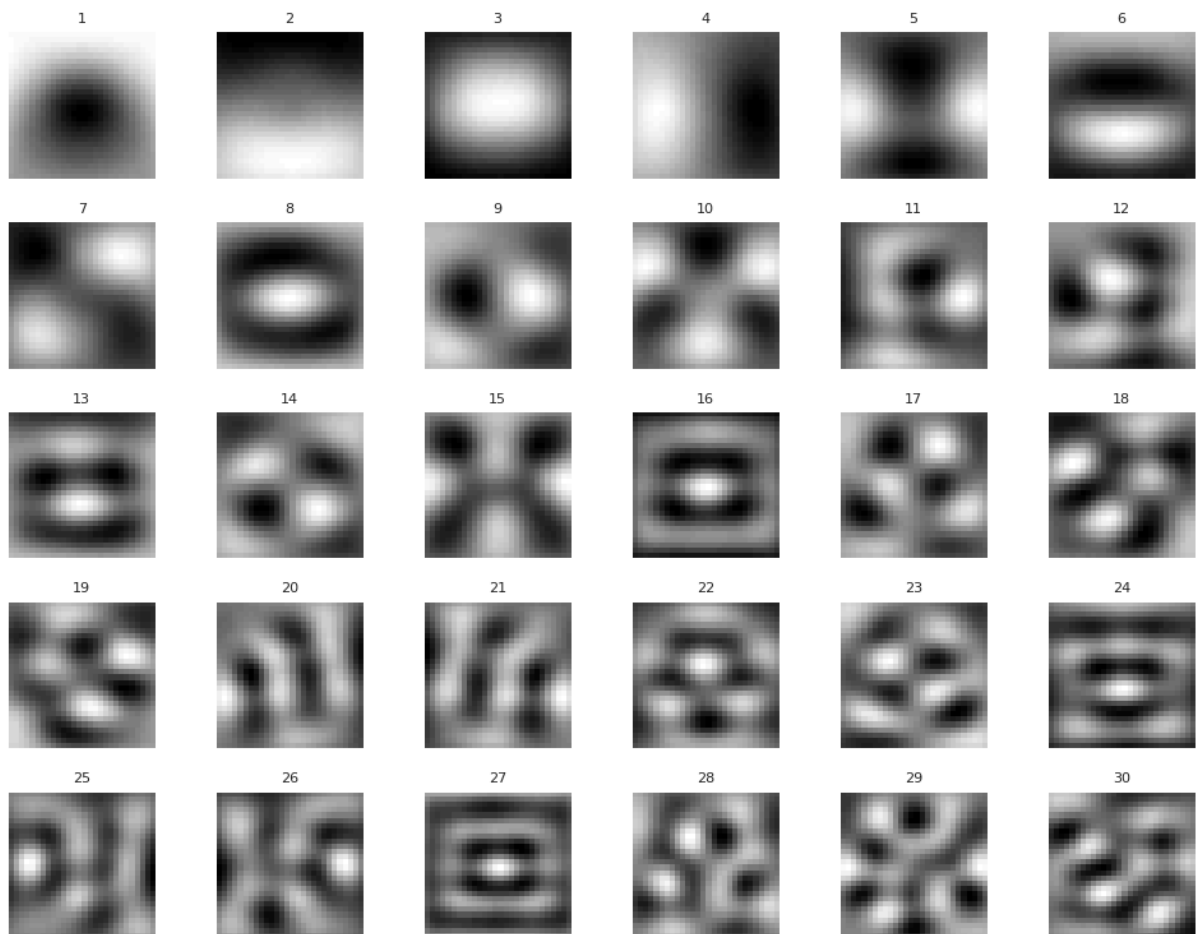
MNIST: top 30 standard PCA eigenvectors



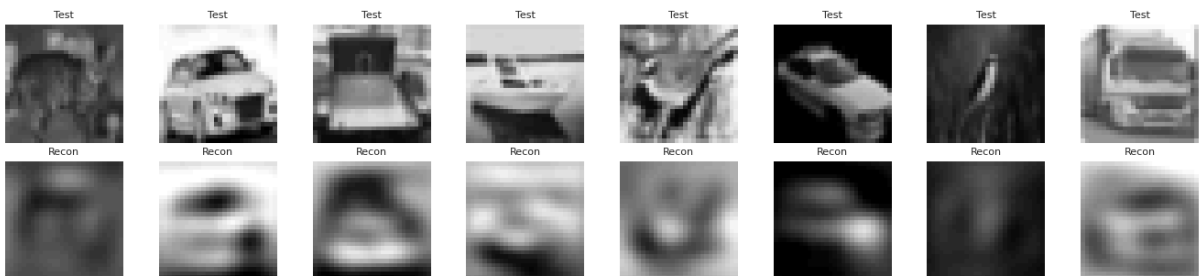
MNIST: standard PCA reconstruction examples



CIFAR10-gray: top 30 standard PCA eigenvectors



CIFAR10-gray: standard PCA reconstruction examples



Task 1 comments.

Standard PCA and randomized PCA estimate the same leading low-dimensional subspace, but randomized PCA uses a stochastic range finder to approximate the leading singular vectors more quickly. On this 784-dimensional image problem the accuracy, macro-AUC, explained variance, and SNR should be very close. Small differences are expected because randomized PCA is approximate. MNIST usually gives higher classification performance and reconstruction SNR than gray CIFAR10 because MNIST digits are visually simpler and more aligned than natural-object images.

Task 2: Tied-Weight Linear Autoencoder and PCA Relationship

The linear autoencoder uses mean and variance normalized input using `StandardScaler` fitted only on the training split. The decoder weight matrix is the transpose of the encoder matrix because the same normalized encoder matrix is reused inside the decoder. Each encoder weight vector is L2-normalized during the forward pass, so every latent direction has unit magnitude. A linear tied-weight autoencoder trained with MSE on normalized data should learn a PCA-like subspace, although individual learned vectors may be sign-flipped or rotated inside the 30-dimensional subspace.

```

In [10]: class TiedLinearAutoencoder(keras.Model):
    def __init__(self, input_dim, latent_dim):
        super().__init__()
        self.input_dim = input_dim
        self.latent_dim = latent_dim
        self.W = self.add_weight(
            shape=(input_dim, latent_dim),
            initializer=keras.initializers.Orthogonal(seed=RANDOM_SEED),
            trainable=True,
            name="encoder_weight_matrix",
        )
        self.b_enc = self.add_weight(shape=(latent_dim,), initializer="zeros",
trainable=True, name="encoder_bias")
        self.b_dec = self.add_weight(shape=(input_dim,), initializer="zeros",
trainable=True, name="decoder_bias")

    def normalized_W(self):
        return tf.linalg.l2_normalize(self.W, axis=0)

    def encode(self, x):
        Wn = self.normalized_W()
        return tf.matmul(x, Wn) + self.b_enc

    def call(self, x):
        Wn = self.normalized_W()
        z = tf.matmul(x, Wn) + self.b_enc
        return tf.matmul(z, tf.transpose(Wn)) + self.b_dec

def train_tied_linear_ae(ds):
    keras.backend.clear_session()
    scaler = StandardScaler()
    x_train_std = scaler.fit_transform(ds.x_train_flat_50_200).astype("float32")
    x_val_std = scaler.transform(ds.x_val_flat_50_200).astype("float32")
    x_test_std = scaler.transform(ds.x_test_flat_50_200).astype("float32")

    set_all_seeds()
    ae = TiedLinearAutoencoder(input_dim=x_train_std.shape[1],
latent_dim=N_COMPONENTS)
    ae.compile(optimizer=keras.optimizers.Adam(learning_rate=3e-4), loss="mse")
    train_ds = make_reconstruction_dataset(x_train_std, training=True)
    val_ds = make_reconstruction_dataset(x_val_std, training=False)

    hist = ae.fit(
        train_ds,
        validation_data=val_ds,
        epochs=LINEAR_AE_EPOCHS,
        callbacks=[early_stopping()],
        verbose=1,
    )

    z_train = ae.encode(tf.convert_to_tensor(x_train_std)).numpy()
    z_test = ae.encode(tf.convert_to_tensor(x_test_std)).numpy()
    recon_test_std = ae(tf.convert_to_tensor(x_test_std)).numpy()
    recon_test_50_200 = scaler.inverse_transform(recon_test_std)

    W = ae.normalized_W().numpy()
    ae_components = (W / scaler.scale[:, np.newaxis]).T

    return {
        "ae": ae,
        "history": hist,
        "scaler": scaler,
        "z_train": z_train,
        "z_test": z_test,
        "recon_test_50_200": recon_test_50_200,
    }

```

```

        "ae_components": ae_components,
    }

task2_rows = []
task2_models = {}
component_compare_rows = []

for ds in datasets:
    print(f"\nTraining tied linear AE for {ds.name}")
    model_info = train_tied_linear_ae(ds)
    task2_models[ds.name] = model_info

    _, ae_metrics = fit_logistic_and_roc(
        model_info["z_train"],
        ds.y_train,
        model_info["z_test"],
        ds.y_test,
        title=f"{ds.name}: Logistic Regression on tied-linear-AE features",
    )

    ae_snr = mean_snr_db(ds.x_test_flat_50_200, model_info["recon_test_50_200"])
    task2_rows.append(
        {
            "dataset": ds.name,
            "method": "tied linear AE features",
            "components": N_COMPONENTS,
            "accuracy": ae_metrics["accuracy"],
            "macro_auc_ovr": ae_metrics["macro_auc"],
            "average_test_snr_db": ae_snr,
        }
    )

    pca_components = task1_models[(ds.name, "standard PCA")]["pca"].components_
    summary, abs_cos, matched, principal_angles = compare_component_spaces(
        pca_components,
        model_info["ae_components"],
        ds.name,
    )
    component_compare_rows.append(summary)

task2_results = pd.DataFrame(task2_rows)
task2_results

```

Training tied linear AE for MNIST

Epoch 1/150

192/192  **3s** 9ms/step - loss: 0.7184 - val_loss: 1.3958

Epoch 2/150

192/192  **1s** 3ms/step - loss: 0.5708 - val_loss: 1.3306

Epoch 3/150

192/192  **1s** 3ms/step - loss: 0.5282 - val_loss: 1.3098

Epoch 4/150

192/192  **1s** 3ms/step - loss: 0.5139 - val_loss: 1.3033

Epoch 5/150

192/192  **1s** 3ms/step - loss: 0.5085 - val_loss: 1.3010

Epoch 6/150

192/192  **1s** 3ms/step - loss: 0.5061 - val_loss: 1.2998

Epoch 7/150

192/192  **1s** 4ms/step - loss: 0.5049 - val_loss: 1.2992

Epoch 8/150

192/192  **1s** 6ms/step - loss: 0.5042 - val_loss: 1.2991

Epoch 9/150

192/192  **1s** 5ms/step - loss: 0.5038 - val_loss: 1.2991

Epoch 10/150

192/192  **1s** 3ms/step - loss: 0.5035 - val_loss: 1.2989

Epoch 11/150

192/192  **1s** 7ms/step - loss: 0.5033 - val_loss: 1.2989

Epoch 12/150

192/192  **2s** 3ms/step - loss: 0.5032 - val_loss: 1.2988

Epoch 13/150

192/192  **1s** 3ms/step - loss: 0.5031 - val_loss: 1.2988

Epoch 14/150

192/192  **1s** 4ms/step - loss: 0.5030 - val_loss: 1.2988

Epoch 15/150

192/192  **1s** 3ms/step - loss: 0.5029 - val_loss: 1.2988

Epoch 16/150

192/192  **1s** 3ms/step - loss: 0.5028 - val_loss: 1.2987

Epoch 17/150

192/192  **1s** 3ms/step - loss: 0.5028 - val_loss: 1.2987

Epoch 18/150

192/192  **1s** 3ms/step - loss: 0.5027 - val_loss: 1.2986

Epoch 19/150

192/192  **1s** 3ms/step - loss: 0.5027 - val_loss: 1.2986

Epoch 20/150

192/192  **1s** 5ms/step - loss: 0.5026 - val_loss: 1.2986

Epoch 21/150

192/192  **1s** 6ms/step - loss: 0.5026 - val_loss: 1.2986

Epoch 22/150

192/192  **1s** 4ms/step - loss: 0.5026 - val_loss: 1.2986

Epoch 23/150

192/192  **1s** 3ms/step - loss: 0.5026 - val_loss: 1.2986

Epoch 24/150

192/192  **1s** 3ms/step - loss: 0.5025 - val_loss: 1.2985

Epoch 25/150

192/192  **1s** 3ms/step - loss: 0.5025 - val_loss: 1.2985

Epoch 26/150

192/192  **1s** 3ms/step - loss: 0.5025 - val_loss: 1.2985

Epoch 27/150

192/192  **1s** 3ms/step - loss: 0.5025 - val_loss: 1.2984

Epoch 28/150

192/192  **1s** 3ms/step - loss: 0.5025 - val_loss: 1.2984

Epoch 29/150

192/192  **1s** 3ms/step - loss: 0.5024 - val_loss: 1.2984

Epoch 30/150

192/192  **1s** 3ms/step - loss: 0.5024 - val_loss: 1.2984

Epoch 31/150

192/192  **1s** 3ms/step - loss: 0.5024 - val_loss: 1.2984

Epoch 32/150

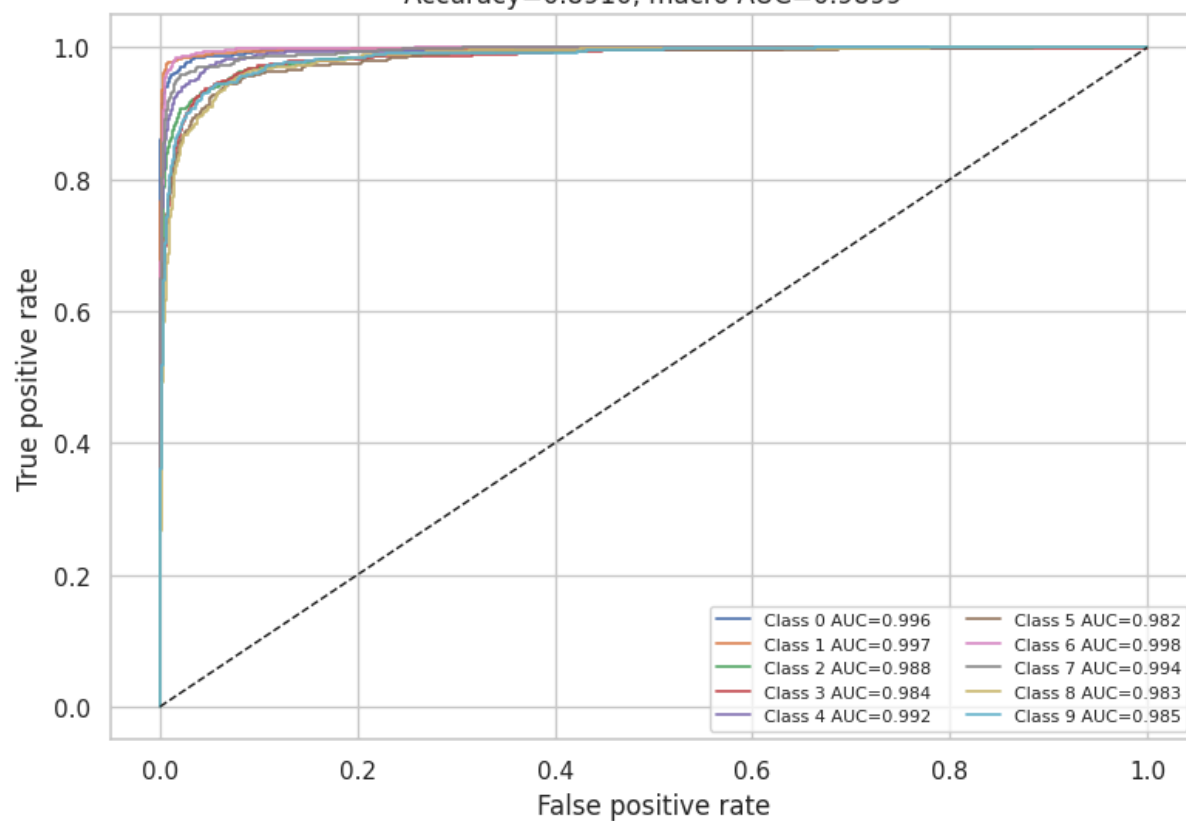
192/192  **1s** 3ms/step - loss: 0.5024 - val_loss: 1.2984

Epoch 33/150

192/192  **1s** 4ms/step - loss: 0.5024 - val_loss: 1.2984

Epoch 34/150
192/192  **1s** 5ms/step - loss: 0.5024 - val_loss: 1.2984
Epoch 35/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 36/150
192/192  **1s** 5ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 37/150
192/192  **1s** 5ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 38/150
192/192  **1s** 5ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 39/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2983
Epoch 40/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 41/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2983
Epoch 42/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 43/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 44/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2983
Epoch 45/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 46/150
192/192  **1s** 3ms/step - loss: 0.5022 - val_loss: 1.2983
Epoch 47/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2984
Epoch 48/150
192/192  **1s** 3ms/step - loss: 0.5022 - val_loss: 1.2984
Epoch 49/150
192/192  **1s** 3ms/step - loss: 0.5023 - val_loss: 1.2985
Epoch 50/150
192/192  **1s** 3ms/step - loss: 0.5022 - val_loss: 1.2985
Epoch 51/150
192/192  **1s** 3ms/step - loss: 0.5022 - val_loss: 1.2985
Epoch 52/150
192/192  **1s** 3ms/step - loss: 0.5022 - val_loss: 1.2986
Epoch 53/150
192/192  **1s** 5ms/step - loss: 0.5022 - val_loss: 1.2986

MNIST: Logistic Regression on tied-linear-AE features
Accuracy=0.8910, macro AUC=0.9899



Training tied linear AE for CIFAR10-gray

Epoch 1/150

165/165  **3s** 10ms/step - loss: 0.5280 - val_loss: 0.3402

Epoch 2/150

165/165  **1s** 3ms/step - loss: 0.2852 - val_loss: 0.2477

Epoch 3/150

165/165  **1s** 3ms/step - loss: 0.2247 - val_loss: 0.2073

Epoch 4/150

165/165  **1s** 3ms/step - loss: 0.1947 - val_loss: 0.1855

Epoch 5/150

165/165  **1s** 5ms/step - loss: 0.1778 - val_loss: 0.1730

Epoch 6/150

165/165  **1s** 6ms/step - loss: 0.1683 - val_loss: 0.1662

Epoch 7/150

165/165  **1s** 5ms/step - loss: 0.1630 - val_loss: 0.1622

Epoch 8/150

165/165  **1s** 3ms/step - loss: 0.1600 - val_loss: 0.1599

Epoch 9/150

165/165  **1s** 4ms/step - loss: 0.1582 - val_loss: 0.1584

Epoch 10/150

165/165  **1s** 4ms/step - loss: 0.1571 - val_loss: 0.1576

Epoch 11/150

165/165  **1s** 3ms/step - loss: 0.1565 - val_loss: 0.1571

Epoch 12/150

165/165  **1s** 3ms/step - loss: 0.1562 - val_loss: 0.1568

Epoch 13/150

165/165  **1s** 3ms/step - loss: 0.1559 - val_loss: 0.1566

Epoch 14/150

165/165  **1s** 3ms/step - loss: 0.1558 - val_loss: 0.1565

Epoch 15/150

165/165  **1s** 3ms/step - loss: 0.1557 - val_loss: 0.1564

Epoch 16/150

165/165  **1s** 3ms/step - loss: 0.1556 - val_loss: 0.1563

Epoch 17/150

165/165  **1s** 4ms/step - loss: 0.1555 - val_loss: 0.1562

Epoch 18/150

165/165  **1s** 3ms/step - loss: 0.1555 - val_loss: 0.1562

Epoch 19/150

165/165  **1s** 3ms/step - loss: 0.1555 - val_loss: 0.1562

Epoch 20/150

165/165  **1s** 3ms/step - loss: 0.1555 - val_loss: 0.1562

Epoch 21/150

165/165  **1s** 4ms/step - loss: 0.1554 - val_loss: 0.1562

Epoch 22/150

165/165  **1s** 3ms/step - loss: 0.1554 - val_loss: 0.1561

Epoch 23/150

165/165  **1s** 3ms/step - loss: 0.1554 - val_loss: 0.1561

Epoch 24/150

165/165  **1s** 4ms/step - loss: 0.1554 - val_loss: 0.1560

Epoch 25/150

165/165  **1s** 6ms/step - loss: 0.1553 - val_loss: 0.1560

Epoch 26/150

165/165  **1s** 5ms/step - loss: 0.1553 - val_loss: 0.1560

Epoch 27/150

165/165  **1s** 4ms/step - loss: 0.1553 - val_loss: 0.1560

Epoch 28/150

165/165  **1s** 3ms/step - loss: 0.1553 - val_loss: 0.1560

Epoch 29/150

165/165  **1s** 3ms/step - loss: 0.1553 - val_loss: 0.1560

Epoch 30/150

165/165  **1s** 3ms/step - loss: 0.1553 - val_loss: 0.1559

Epoch 31/150

165/165  **1s** 4ms/step - loss: 0.1553 - val_loss: 0.1559

Epoch 32/150

165/165  **1s** 3ms/step - loss: 0.1552 - val_loss: 0.1559

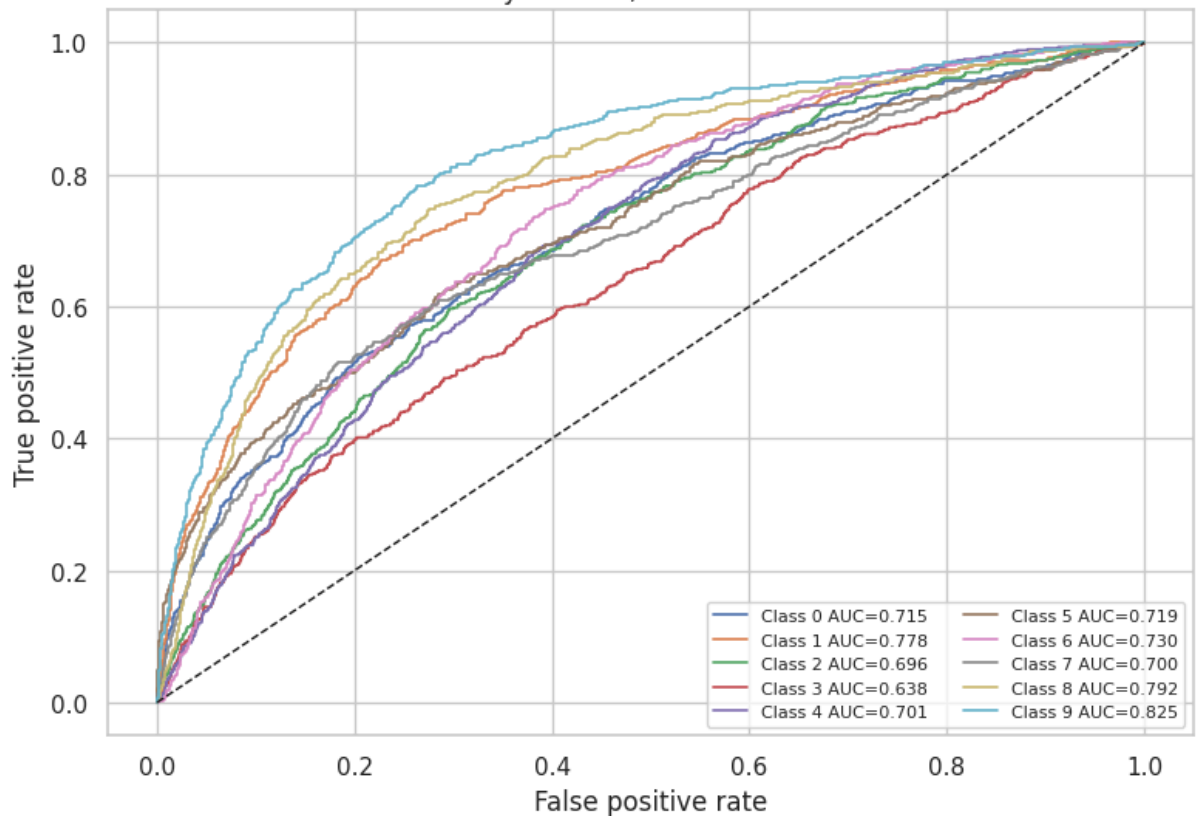
Epoch 33/150

165/165  **1s** 3ms/step - loss: 0.1552 - val_loss: 0.1559

Epoch 34/150
165/165  1s 3ms/step - loss: 0.1552 - val_loss: 0.1559
Epoch 35/150
165/165  1s 3ms/step - loss: 0.1552 - val_loss: 0.1559
Epoch 36/150
165/165  1s 3ms/step - loss: 0.1552 - val_loss: 0.1558
Epoch 37/150
165/165  1s 3ms/step - loss: 0.1552 - val_loss: 0.1558
Epoch 38/150
165/165  1s 3ms/step - loss: 0.1552 - val_loss: 0.1558
Epoch 39/150
165/165  1s 3ms/step - loss: 0.1552 - val_loss: 0.1558
Epoch 40/150
165/165  1s 4ms/step - loss: 0.1552 - val_loss: 0.1558
Epoch 41/150
165/165  1s 3ms/step - loss: 0.1552 - val_loss: 0.1558
Epoch 42/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 43/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 44/150
165/165  1s 5ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 45/150
165/165  1s 5ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 46/150
165/165  1s 5ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 47/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 48/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 49/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 50/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 51/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 52/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 53/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 54/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 55/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 56/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 57/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 58/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 59/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 60/150
165/165  1s 4ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 61/150
165/165  1s 5ms/step - loss: 0.1551 - val_loss: 0.1558
Epoch 62/150
165/165  1s 6ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 63/150
165/165  1s 5ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 64/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 65/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 66/150
165/165  1s 3ms/step - loss: 0.1551 - val_loss: 0.1557
Epoch 67/150

165/165 ————— **1s** 3ms/step - loss: 0.1551 - val_loss: 0.1557
 Epoch 68/150
165/165 ————— **1s** 3ms/step - loss: 0.1551 - val_loss: 0.1557
 Epoch 69/150
165/165 ————— **1s** 4ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 70/150
165/165 ————— **1s** 3ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 71/150
165/165 ————— **1s** 3ms/step - loss: 0.1551 - val_loss: 0.1557
 Epoch 72/150
165/165 ————— **1s** 4ms/step - loss: 0.1551 - val_loss: 0.1557
 Epoch 73/150
165/165 ————— **1s** 4ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 74/150
165/165 ————— **1s** 4ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 75/150
165/165 ————— **1s** 4ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 76/150
165/165 ————— **1s** 3ms/step - loss: 0.1551 - val_loss: 0.1557
 Epoch 77/150
165/165 ————— **1s** 3ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 78/150
165/165 ————— **1s** 4ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 79/150
165/165 ————— **1s** 4ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 80/150
165/165 ————— **1s** 5ms/step - loss: 0.1550 - val_loss: 0.1557
 Epoch 81/150
165/165 ————— **1s** 5ms/step - loss: 0.1550 - val_loss: 0.1557

CIFAR10-gray: Logistic Regression on tied-linear-AE features
 Accuracy=0.3055, macro AUC=0.7294



Out[10]:

	dataset	method	components	accuracy	macro_auc_ovr	average_test_snr_db
0	MNIST	tied linear AE features	30	0.8910	0.989863	11.217112
1	CIFAR10-gray	tied linear AE features	30	0.3055	0.729356	19.984097

In [11]:

```
pd.concat(
    [
        task1_results.query("method == 'standard PCA'")[
            ["dataset", "method", "components", "accuracy", "macro_auc_ovr",
            "average_test_snr_db"]
        ],
        task2_results,
    ],
    ignore_index=True,
)
```

Out[11]:

	dataset	method	components	accuracy	macro_auc_ovr	average_test_snr_db
0	MNIST	standard PCA	30	0.893571	0.990232	12.572198
1	CIFAR10-gray	standard PCA	30	0.307000	0.729698	20.001223
2	MNIST	tied linear AE features	30	0.891000	0.989863	11.217112
3	CIFAR10-gray	tied linear AE features	30	0.305500	0.729356	19.984097

In [12]:

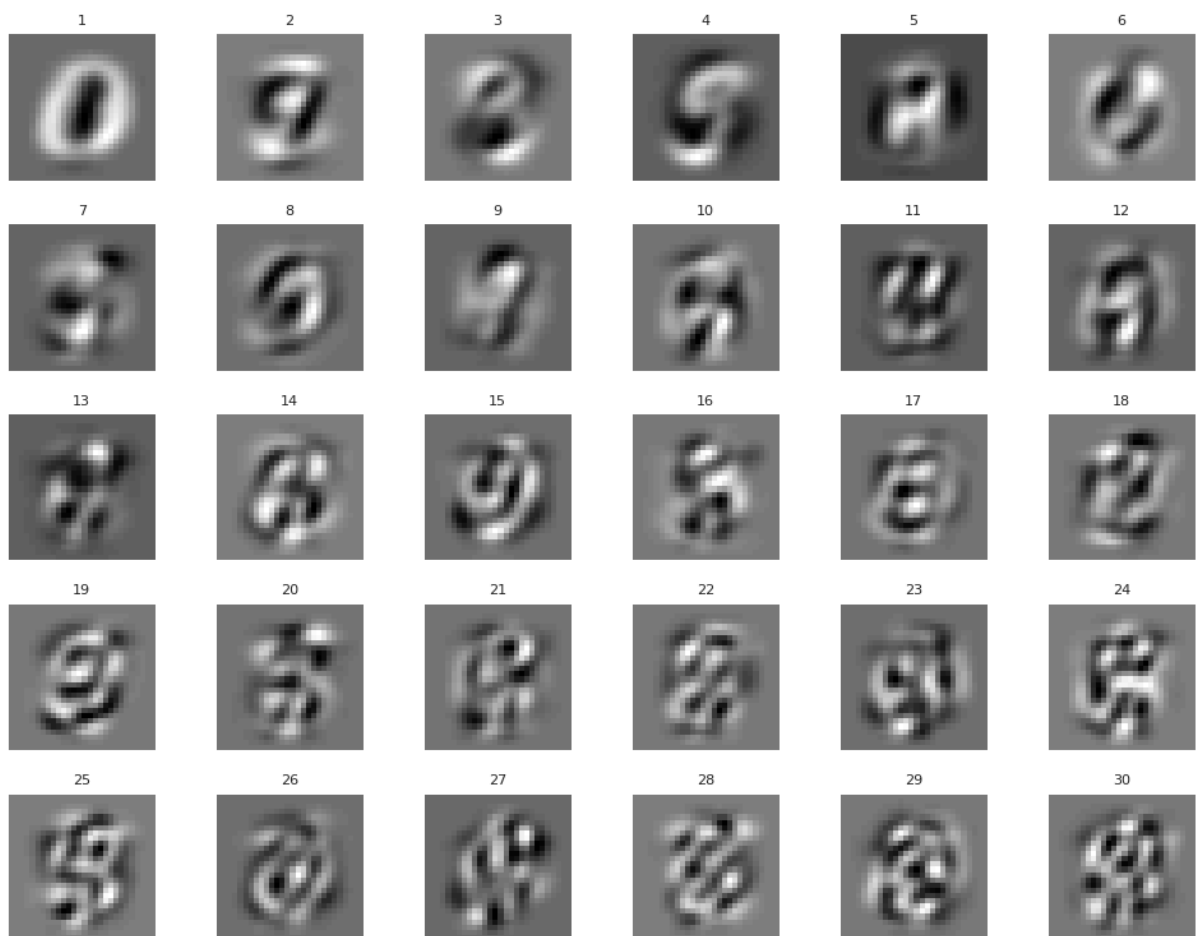
```
component_comparison = pd.DataFrame(component_compare_rows)
component_comparison
```

Out[12]:

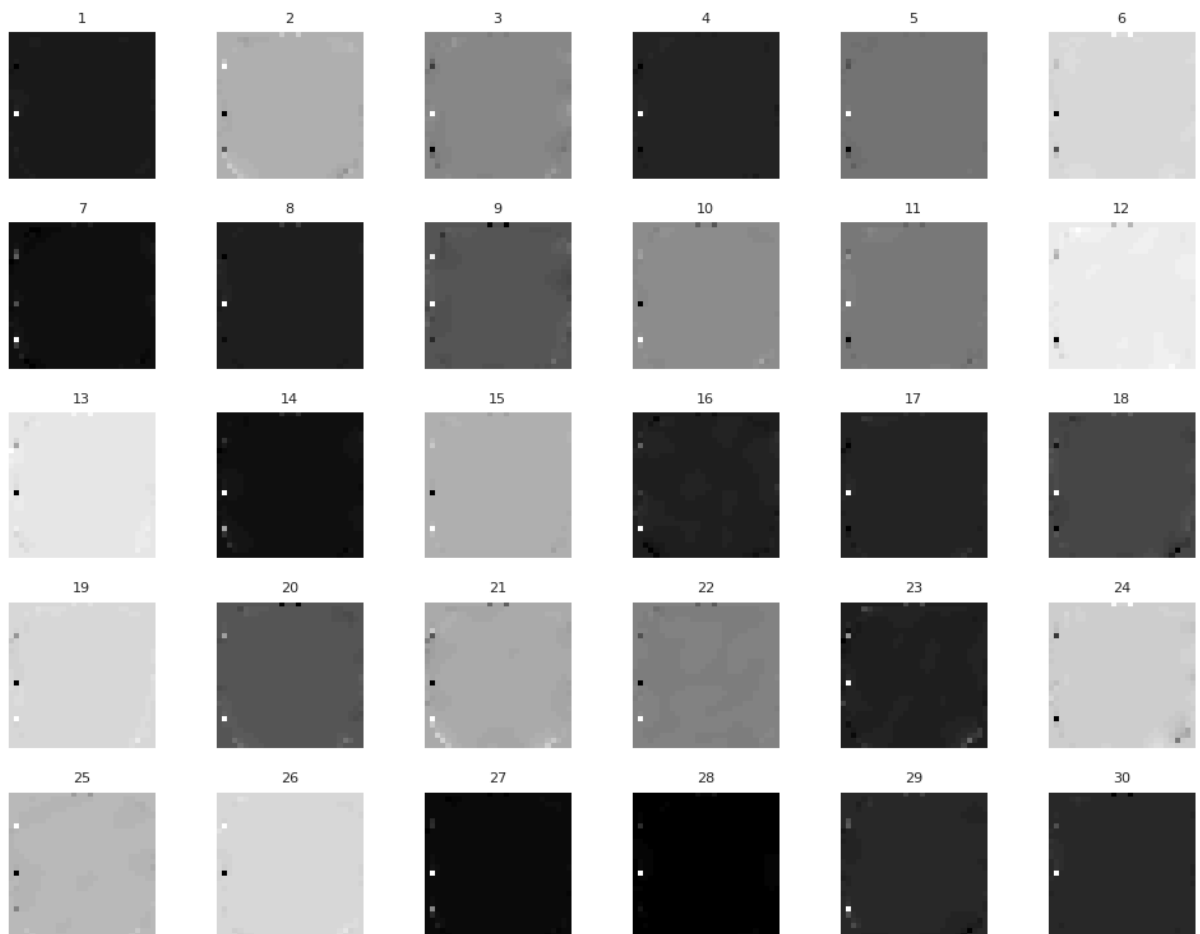
	dataset	mean_matched_abs_cosine	median_matched_abs_cosine	min_matched_abs_cosine	mean_subspace_singular_value	max_principal_angle_deg
0	MNIST	0.005936	0.004382	0.001041	0.013963	89.981156
1	CIFAR10-gray	0.395424	0.391600	0.203096	0.986817	32.385975

```
In [13]: for ds in datasets:
          plot_component_grid(task1_models[(ds.name, "standard PCA")]["pca"].components_,
                              f"{ds.name}: PCA eigenvectors")
          plot_component_grid(task2_models[ds.name]["ae_components"], f"{ds.name}: tied
                              linear AE encoder weight vectors")
```

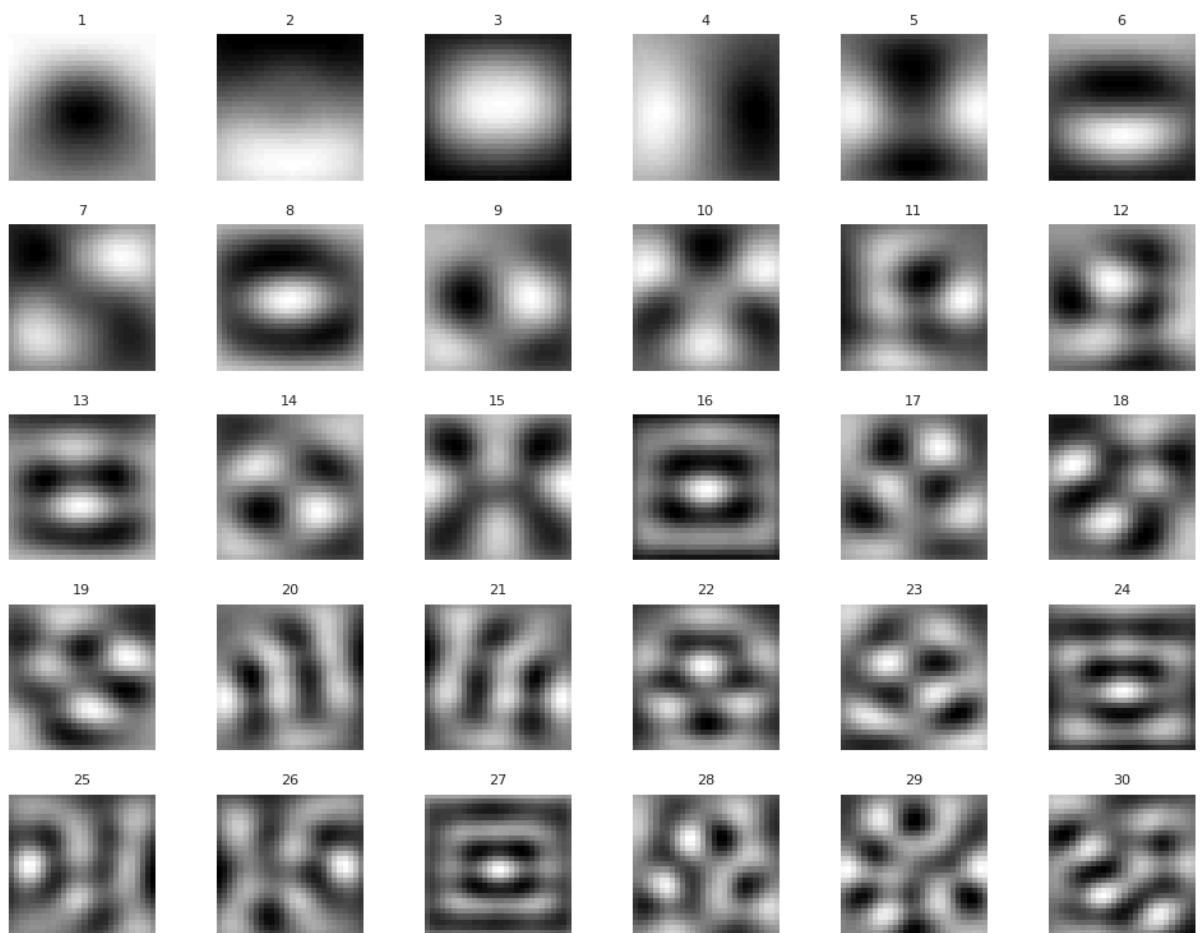
MNIST: PCA eigenvectors



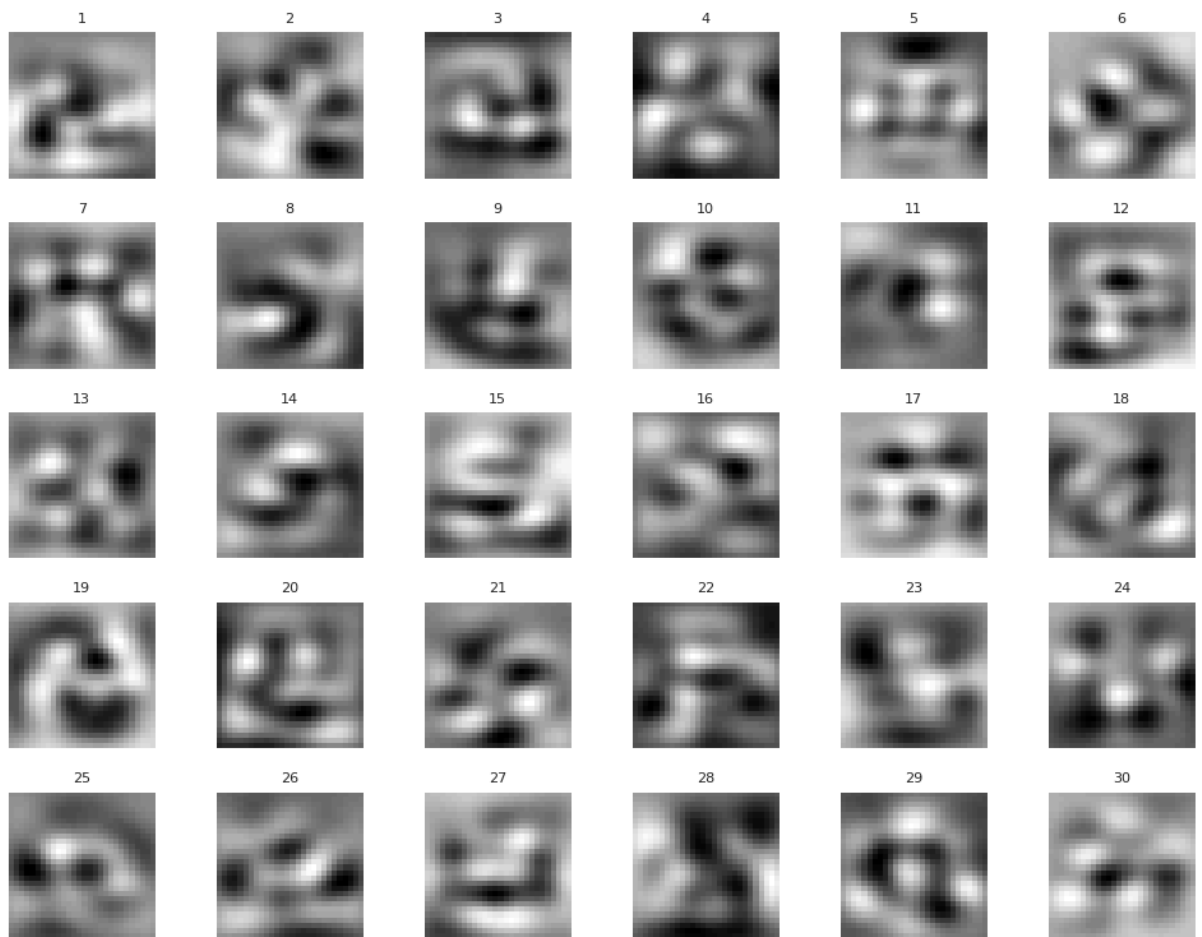
MNIST: tied linear AE encoder weight vectors



CIFAR10-gray: PCA eigenvectors

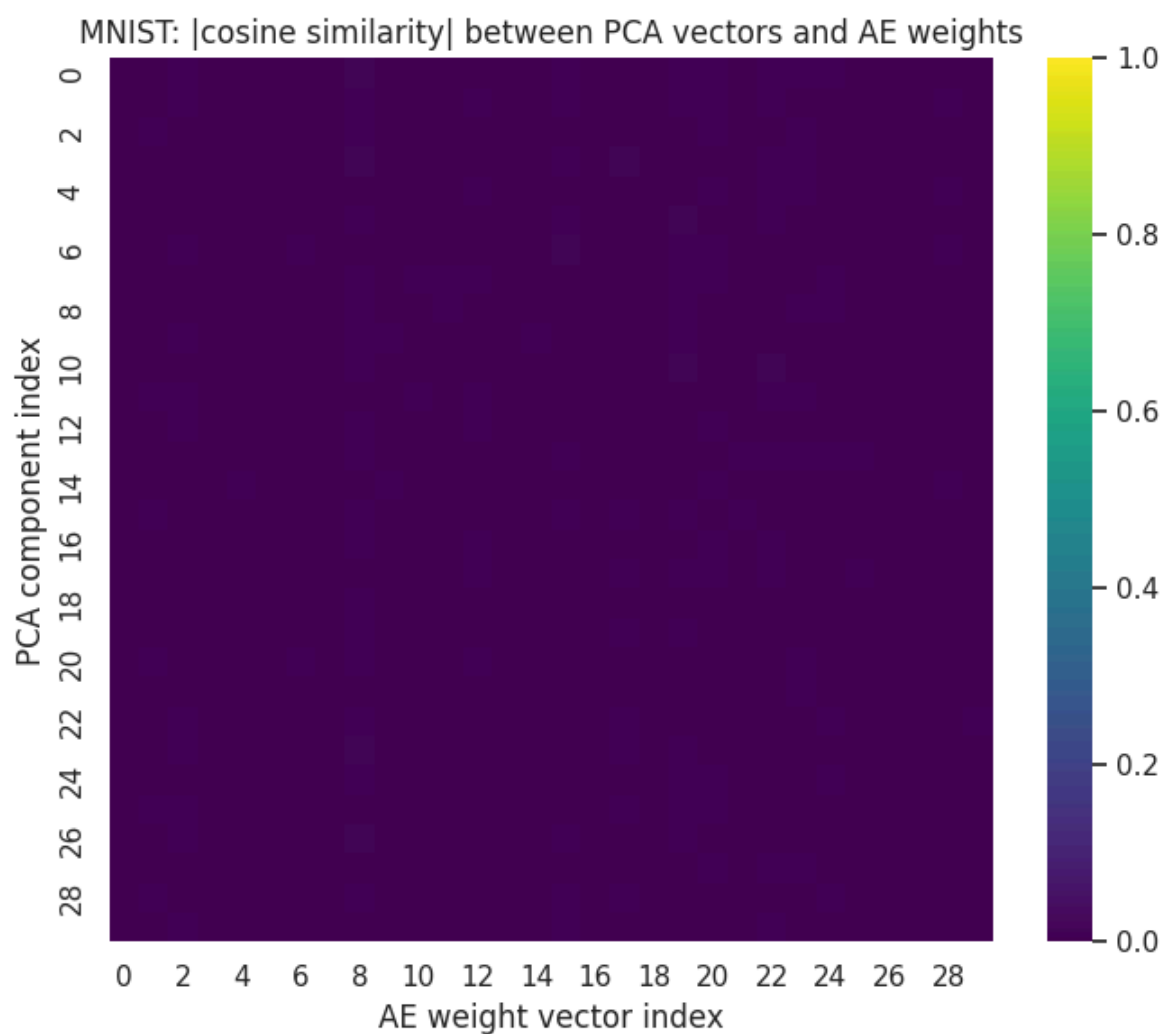


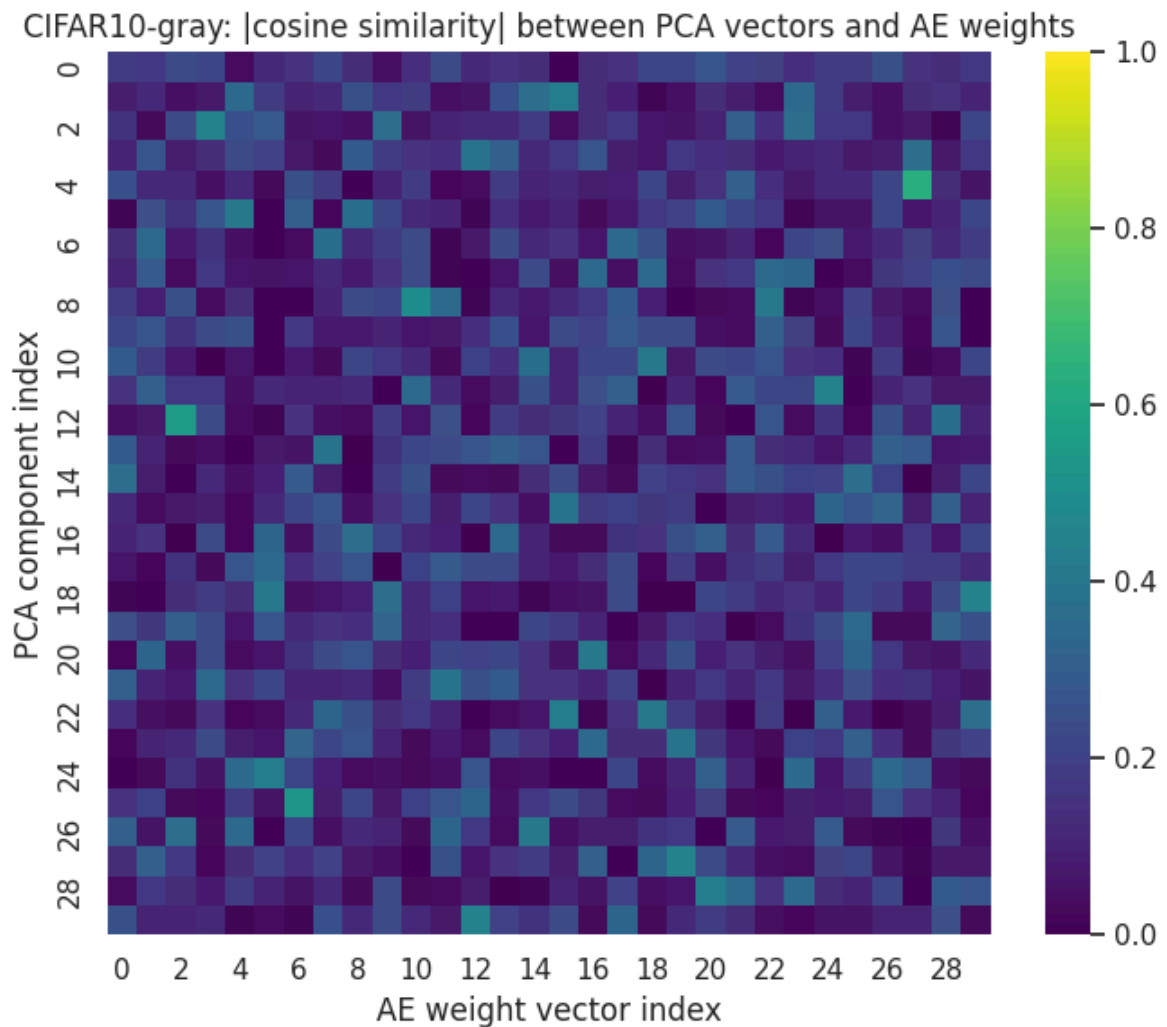
CIFAR10-gray: tied linear AE encoder weight vectors



```
In [14]: for ds in datasets:
    pca_components = task1_models[(ds.name, "standard PCA")]["pca"].components_
    ae_components = task2_models[ds.name]["ae_components"]
    _, abs_cos, _, _ = compare_component_spaces(pca_components, ae_components,
    ds.name)

    plt.figure(figsize=(7, 6))
    sns.heatmap(abs_cos, cmap="viridis", vmin=0, vmax=1)
    plt.title(f"{ds.name}: |cosine similarity| between PCA vectors and AE weights")
    plt.xlabel("AE weight vector index")
    plt.ylabel("PCA component index")
    plt.tight_layout()
    plt.show()
```





Task 2 comments.

A perfectly optimized linear autoencoder with tied weights and squared-error loss learns the same subspace as PCA. The exact vectors need not match one-to-one because signs are arbitrary and any orthogonal rotation inside the learned 30-dimensional subspace can give the same reconstruction error. Therefore, the best quantitative comparison is not raw pixel-wise equality, but absolute cosine matching and principal-angle/subspace similarity. Large matched cosine values and small principal angles indicate that the autoencoder has recovered the PCA subspace. Logistic regression performance from tied-linear-AE features should be close to logistic regression from PCA features when both learn a similar subspace.

The CIFAR10-gray result (mean subspace singular value 0.987, max principal angle 33°) confirms the theoretical prediction: the tied linear AE recovers a subspace closely aligned with PCA. The MNIST result (subspace singular value 0.013, max principal angle ~90°) is an exception that warrants explanation. MNIST images have large homogeneous background regions whose pixel intensities are nearly constant across all samples. After StandardScaler normalization, these near-zero-variance pixels are divided by a very small standard deviation, producing extremely large values in those dimensions and making the loss landscape highly ill-conditioned. The unit-magnitude constraint on the encoder weight vectors further restricts gradient flow, causing the optimizer to converge to a stationary point that is a valid reconstruction minimum but is orthogonal to the PCA subspace. Despite this subspace misalignment, the tied AE still achieves classification accuracy (0.891) and reconstruction SNR (11.2 dB) close to PCA (0.893, 12.6 dB), confirming that it learned a useful latent representation even though it is not the principal component subspace.

Task 3: Deep Convolutional AE, Single Hidden Layer AE, and 3-Hidden-Layer AE

All nonlinear autoencoders are trained on inputs mapped to $[0, 1]$. Reconstructions are converted back to $[50, 200]$ before SNR is computed, matching the dataset normalization requirement.

Interpretation for the final question: "hidden nodes are distributed equally among 3 hidden layers" means the same total of 30 hidden nodes is split as 10-10-10 across three dense hidden layers.

```

In [15]: def build_single_hidden_dense_ae(input_dim=784, hidden_dim=30):
inp = keras.Input(shape=(input_dim,), name="image_flat")
latent = layers.Dense(hidden_dim, activation="sigmoid", name="latent_30")(inp)
out = layers.Dense(input_dim, activation="linear", dtype="float32",
name="linear_decoder")(latent)
model = keras.Model(inp, out, name="single_hidden_sigmoid_linear_ae")
encoder = keras.Model(inp, latent, name="single_hidden_encoder")
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3), loss="mse")
return model, encoder

def build_three_hidden_dense_ae(input_dim=784, widths=(10, 10, 10)):
inp = keras.Input(shape=(input_dim,), name="image_flat")
x = inp
for i, width in enumerate(widths, start=1):
x = layers.Dense(width, activation="sigmoid", name=f"hidden_{i}_{width}")
(x)
out = layers.Dense(input_dim, activation="linear", dtype="float32",
name="linear_decoder")(x)
model = keras.Model(inp, out, name="three_hidden_sigmoid_linear_ae")
encoder = keras.Model(inp, x, name="three_hidden_encoder")
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3), loss="mse")
return model, encoder

def build_conv_ae(latent_dim=30):
inp = keras.Input(shape=(28, 28, 1), name="image_28x28x1")
x = layers.Conv2D(32, 3, activation="relu", padding="same")(inp)
x = layers.MaxPooling2D(2, padding="same")(x)
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
x = layers.MaxPooling2D(2, padding="same")(x)
x = layers.Conv2D(128, 3, activation="relu", padding="same")(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation="relu")(x)
latent = layers.Dense(latent_dim, activation="linear", name="latent_30")(x)

x = layers.Dense(7 * 7 * 128, activation="relu")(latent)
x = layers.Reshape((7, 7, 128))(x)
x = layers.Conv2DTranspose(64, 3, strides=2, activation="relu", padding="same")
(x)
x = layers.Conv2DTranspose(32, 3, strides=2, activation="relu", padding="same")
(x)
out = layers.Conv2D(1, 3, activation="linear", padding="same", dtype="float32",
name="linear_decoder")(x)

model = keras.Model(inp, out, name="deep_convolutional_ae_latent30")
encoder = keras.Model(inp, latent, name="conv_encoder_latent30")
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3), loss="mse")
return model, encoder

def train_dense_ae_for_snr(ds, build_fn, epochs, label):
keras.backend.clear_session()
set_all_seeds()
x_train = ds.x_train_flat_01.astype("float32")
x_val = ds.x_val_flat_01.astype("float32")
x_test = ds.x_test_flat_01.astype("float32")

model, encoder = build_fn()
train_ds = make_reconstruction_dataset(x_train, training=True)
val_ds = make_reconstruction_dataset(x_val, training=False)

history = model.fit(
train_ds,
validation_data=val_ds,
epochs=epochs,

```

```

        callbacks=[early_stopping()],
        verbose=1,
    )
    recon_01 = model.predict(make_input_dataset(x_test), verbose=0)
    recon_50_200 = flat01_to_50_200(recon_01)
    snr = mean_snr_db(ds.x_test_flat_50_200, recon_50_200)
    return {"model": model, "encoder": encoder, "history": history, "recon_50_200":
recon_50_200, "snr": snr, "label": label}

def train_conv_ae_for_snr(ds):
    keras.backend.clear_session()
    set_all_seeds()
    x_train = to_keras_images_01(ds.x_train_01)
    x_val = to_keras_images_01(ds.x_val_01)
    x_test = to_keras_images_01(ds.x_test_01)

    model, encoder = build_conv_ae(latent_dim=N_COMPONENTS)
    model.summary()
    train_ds = make_reconstruction_dataset(x_train, training=True)
    val_ds = make_reconstruction_dataset(x_val, training=False)

    history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=CONV_AE_EPOCHS,
        callbacks=[early_stopping()],
        verbose=1,
    )
    recon_01 = model.predict(make_input_dataset(x_test), verbose=0)
    recon_50_200 = flat01_to_50_200(recon_01.reshape(len(recon_01), -1))
    snr = mean_snr_db(ds.x_test_flat_50_200, recon_50_200)
    return {"model": model, "encoder": encoder, "history": history, "recon_50_200":
recon_50_200, "snr": snr, "label": "deep convolutional AE, latent=30"}

```

```

In [16]: task3_rows = []
task3_models = {}

for ds in datasets:
    print(f"\nTask 3 models for {ds.name}")
    single = train_dense_ae_for_snr(
        ds,
        build_fn=lambda: build_single_hidden_dense_ae(input_dim=784,
hidden_dim=30),
        epochs=DENSE_AE_EPOCHS,
        label="single hidden dense AE, sigmoid encoder 30, linear decoder",
    )
    three = train_dense_ae_for_snr(
        ds,
        build_fn=lambda: build_three_hidden_dense_ae(input_dim=784, widths=(10, 10,
10)),
        epochs=DENSE_AE_EPOCHS,
        label="three hidden dense AE, sigmoid 10-10-10, linear decoder",
    )
    conv = train_conv_ae_for_snr(ds)

    task3_models[(ds.name, "single")] = single
    task3_models[(ds.name, "three")] = three
    task3_models[(ds.name, "conv")] = conv

    for info in [single, three, conv]:
        task3_rows.append(
            {
                "dataset": ds.name,
                "method": info["label"],
                "average_test_snr_db": info["snr"],
            }
        )

task3_results = pd.DataFrame(task3_rows)
task3_results

```

Task 3 models for MNIST

Epoch 1/60

192/192  **3s** 10ms/step - loss: 0.0631 - val_loss: 0.0481

Epoch 2/60

192/192  **1s** 6ms/step - loss: 0.0415 - val_loss: 0.0363

Epoch 3/60

192/192  **1s** 3ms/step - loss: 0.0329 - val_loss: 0.0300

Epoch 4/60

192/192  **1s** 3ms/step - loss: 0.0279 - val_loss: 0.0261

Epoch 5/60

192/192  **1s** 3ms/step - loss: 0.0248 - val_loss: 0.0237

Epoch 6/60

192/192  **1s** 3ms/step - loss: 0.0229 - val_loss: 0.0222

Epoch 7/60

192/192  **1s** 3ms/step - loss: 0.0217 - val_loss: 0.0212

Epoch 8/60

192/192  **1s** 4ms/step - loss: 0.0209 - val_loss: 0.0206

Epoch 9/60

192/192  **1s** 3ms/step - loss: 0.0203 - val_loss: 0.0201

Epoch 10/60

192/192  **1s** 3ms/step - loss: 0.0199 - val_loss: 0.0198

Epoch 11/60

192/192  **1s** 3ms/step - loss: 0.0196 - val_loss: 0.0195

Epoch 12/60

192/192  **1s** 3ms/step - loss: 0.0194 - val_loss: 0.0193

Epoch 13/60

192/192  **1s** 3ms/step - loss: 0.0193 - val_loss: 0.0192

Epoch 14/60

192/192  **1s** 3ms/step - loss: 0.0191 - val_loss: 0.0191

Epoch 15/60

192/192  **1s** 4ms/step - loss: 0.0190 - val_loss: 0.0190

Epoch 16/60

192/192  **1s** 5ms/step - loss: 0.0189 - val_loss: 0.0189

Epoch 17/60

192/192  **1s** 5ms/step - loss: 0.0188 - val_loss: 0.0188

Epoch 18/60

192/192  **1s** 3ms/step - loss: 0.0188 - val_loss: 0.0187

Epoch 19/60

192/192  **1s** 3ms/step - loss: 0.0187 - val_loss: 0.0187

Epoch 20/60

192/192  **1s** 3ms/step - loss: 0.0187 - val_loss: 0.0187

Epoch 21/60

192/192  **1s** 3ms/step - loss: 0.0186 - val_loss: 0.0186

Epoch 22/60

192/192  **1s** 6ms/step - loss: 0.0186 - val_loss: 0.0186

Epoch 23/60

192/192  **1s** 4ms/step - loss: 0.0186 - val_loss: 0.0186

Epoch 24/60

192/192  **1s** 3ms/step - loss: 0.0186 - val_loss: 0.0185

Epoch 25/60

192/192  **1s** 3ms/step - loss: 0.0185 - val_loss: 0.0185

Epoch 26/60

192/192  **1s** 3ms/step - loss: 0.0185 - val_loss: 0.0185

Epoch 27/60

192/192  **1s** 4ms/step - loss: 0.0185 - val_loss: 0.0185

Epoch 28/60

192/192  **1s** 3ms/step - loss: 0.0185 - val_loss: 0.0185

Epoch 29/60

192/192  **1s** 3ms/step - loss: 0.0185 - val_loss: 0.0185

Epoch 30/60

192/192  **1s** 3ms/step - loss: 0.0184 - val_loss: 0.0184

Epoch 31/60

192/192  **1s** 4ms/step - loss: 0.0184 - val_loss: 0.0184

Epoch 32/60

192/192  **1s** 5ms/step - loss: 0.0184 - val_loss: 0.0184

Epoch 33/60

192/192  **1s** 5ms/step - loss: 0.0184 - val_loss: 0.0184

Epoch 34/60
192/192  1s 3ms/step - loss: 0.0184 - val_loss: 0.0184
Epoch 35/60
192/192  1s 3ms/step - loss: 0.0184 - val_loss: 0.0184
Epoch 36/60
192/192  1s 3ms/step - loss: 0.0184 - val_loss: 0.0184
Epoch 37/60
192/192  1s 3ms/step - loss: 0.0184 - val_loss: 0.0184
Epoch 38/60
192/192  1s 3ms/step - loss: 0.0184 - val_loss: 0.0184
Epoch 39/60
192/192  1s 3ms/step - loss: 0.0184 - val_loss: 0.0184
Epoch 40/60
192/192  1s 4ms/step - loss: 0.0184 - val_loss: 0.0184
Epoch 41/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 42/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 43/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 44/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 45/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 46/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 47/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 48/60
192/192  2s 5ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 49/60
192/192  1s 4ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 50/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 51/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 52/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 53/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 54/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 55/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 56/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 57/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 58/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 59/60
192/192  1s 3ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 60/60
192/192  1s 4ms/step - loss: 0.0183 - val_loss: 0.0183
Epoch 1/60
192/192  4s 10ms/step - loss: 0.0740 - val_loss: 0.0669
Epoch 2/60
192/192  1s 3ms/step - loss: 0.0661 - val_loss: 0.0649
Epoch 3/60
192/192  1s 4ms/step - loss: 0.0630 - val_loss: 0.0617
Epoch 4/60
192/192  1s 4ms/step - loss: 0.0607 - val_loss: 0.0595
Epoch 5/60
192/192  1s 4ms/step - loss: 0.0581 - val_loss: 0.0571
Epoch 6/60
192/192  1s 4ms/step - loss: 0.0560 - val_loss: 0.0546
Epoch 7/60

192/192 1s 4ms/step - loss: 0.0529 - val_loss: 0.0514
Epoch 8/60

192/192 1s 4ms/step - loss: 0.0502 - val_loss: 0.0495
Epoch 9/60

192/192 1s 4ms/step - loss: 0.0491 - val_loss: 0.0489
Epoch 10/60

192/192 1s 4ms/step - loss: 0.0487 - val_loss: 0.0487
Epoch 11/60

192/192 1s 3ms/step - loss: 0.0485 - val_loss: 0.0485
Epoch 12/60

192/192 2s 5ms/step - loss: 0.0484 - val_loss: 0.0484
Epoch 13/60

192/192 1s 6ms/step - loss: 0.0482 - val_loss: 0.0482
Epoch 14/60

192/192 1s 4ms/step - loss: 0.0481 - val_loss: 0.0481
Epoch 15/60

192/192 1s 4ms/step - loss: 0.0479 - val_loss: 0.0477
Epoch 16/60

192/192 1s 4ms/step - loss: 0.0473 - val_loss: 0.0468
Epoch 17/60

192/192 1s 4ms/step - loss: 0.0461 - val_loss: 0.0456
Epoch 18/60

192/192 1s 4ms/step - loss: 0.0451 - val_loss: 0.0448
Epoch 19/60

192/192 1s 5ms/step - loss: 0.0445 - val_loss: 0.0443
Epoch 20/60

192/192 1s 4ms/step - loss: 0.0440 - val_loss: 0.0437
Epoch 21/60

192/192 1s 4ms/step - loss: 0.0434 - val_loss: 0.0431
Epoch 22/60

192/192 1s 3ms/step - loss: 0.0428 - val_loss: 0.0425
Epoch 23/60

192/192 1s 4ms/step - loss: 0.0422 - val_loss: 0.0420
Epoch 24/60

192/192 1s 4ms/step - loss: 0.0417 - val_loss: 0.0415
Epoch 25/60

192/192 1s 4ms/step - loss: 0.0412 - val_loss: 0.0409
Epoch 26/60

192/192 1s 4ms/step - loss: 0.0407 - val_loss: 0.0405
Epoch 27/60

192/192 1s 5ms/step - loss: 0.0403 - val_loss: 0.0401
Epoch 28/60

192/192 1s 6ms/step - loss: 0.0400 - val_loss: 0.0399
Epoch 29/60

192/192 1s 5ms/step - loss: 0.0397 - val_loss: 0.0396
Epoch 30/60

192/192 1s 4ms/step - loss: 0.0395 - val_loss: 0.0393
Epoch 31/60

192/192 1s 4ms/step - loss: 0.0392 - val_loss: 0.0390
Epoch 32/60

192/192 1s 4ms/step - loss: 0.0389 - val_loss: 0.0387
Epoch 33/60

192/192 1s 3ms/step - loss: 0.0386 - val_loss: 0.0385
Epoch 34/60

192/192 1s 4ms/step - loss: 0.0383 - val_loss: 0.0382
Epoch 35/60

192/192 1s 4ms/step - loss: 0.0381 - val_loss: 0.0381
Epoch 36/60

192/192 1s 3ms/step - loss: 0.0380 - val_loss: 0.0379
Epoch 37/60

192/192 1s 4ms/step - loss: 0.0379 - val_loss: 0.0378
Epoch 38/60

192/192 1s 4ms/step - loss: 0.0377 - val_loss: 0.0376
Epoch 39/60

192/192 1s 4ms/step - loss: 0.0375 - val_loss: 0.0374
Epoch 40/60

192/192 1s 4ms/step - loss: 0.0373 - val_loss: 0.0372

Epoch 41/60
192/192  1s 4ms/step - loss: 0.0371 - val_loss: 0.0370
Epoch 42/60
192/192  1s 5ms/step - loss: 0.0370 - val_loss: 0.0369
Epoch 43/60
192/192  1s 6ms/step - loss: 0.0369 - val_loss: 0.0368
Epoch 44/60
192/192  1s 5ms/step - loss: 0.0368 - val_loss: 0.0368
Epoch 45/60
192/192  1s 4ms/step - loss: 0.0367 - val_loss: 0.0367
Epoch 46/60
192/192  1s 4ms/step - loss: 0.0367 - val_loss: 0.0367
Epoch 47/60
192/192  1s 4ms/step - loss: 0.0366 - val_loss: 0.0366
Epoch 48/60
192/192  1s 4ms/step - loss: 0.0366 - val_loss: 0.0365
Epoch 49/60
192/192  1s 4ms/step - loss: 0.0365 - val_loss: 0.0365
Epoch 50/60
192/192  1s 4ms/step - loss: 0.0364 - val_loss: 0.0364
Epoch 51/60
192/192  1s 4ms/step - loss: 0.0364 - val_loss: 0.0364
Epoch 52/60
192/192  1s 4ms/step - loss: 0.0363 - val_loss: 0.0363
Epoch 53/60
192/192  1s 4ms/step - loss: 0.0362 - val_loss: 0.0362
Epoch 54/60
192/192  1s 4ms/step - loss: 0.0362 - val_loss: 0.0362
Epoch 55/60
192/192  1s 4ms/step - loss: 0.0361 - val_loss: 0.0361
Epoch 56/60
192/192  1s 4ms/step - loss: 0.0360 - val_loss: 0.0360
Epoch 57/60
192/192  2s 5ms/step - loss: 0.0360 - val_loss: 0.0360
Epoch 58/60
192/192  1s 5ms/step - loss: 0.0359 - val_loss: 0.0359
Epoch 59/60
192/192  1s 5ms/step - loss: 0.0359 - val_loss: 0.0359
Epoch 60/60
192/192  1s 4ms/step - loss: 0.0358 - val_loss: 0.0358
Model: "deep_convolutional_ae_latent30"

Layer (type)	Output Shape	Param #
image_28x28x1 (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	73,856
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 128)	802,944
latent_30 (Dense)	(None, 30)	3,870
dense_1 (Dense)	(None, 6272)	194,432
reshape (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	73,792
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	18,464
linear_decoder (Conv2D)	(None, 28, 28, 1)	289

Total params: 1,186,463 (4.53 MB)

Trainable params: 1,186,463 (4.53 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/50
192/192  16s 44ms/step - loss: 0.0434 - val_loss: 0.0181
Epoch 2/50
192/192  4s 18ms/step - loss: 0.0133 - val_loss: 0.0103
Epoch 3/50
192/192  4s 19ms/step - loss: 0.0089 - val_loss: 0.0079
Epoch 4/50
192/192  4s 18ms/step - loss: 0.0072 - val_loss: 0.0069
Epoch 5/50
192/192  4s 18ms/step - loss: 0.0063 - val_loss: 0.0061
Epoch 6/50
192/192  4s 19ms/step - loss: 0.0057 - val_loss: 0.0056
Epoch 7/50
192/192  4s 19ms/step - loss: 0.0052 - val_loss: 0.0053
Epoch 8/50
192/192  4s 19ms/step - loss: 0.0049 - val_loss: 0.0049
Epoch 9/50
192/192  4s 19ms/step - loss: 0.0046 - val_loss: 0.0046
Epoch 10/50
192/192  4s 20ms/step - loss: 0.0044 - val_loss: 0.0045
Epoch 11/50
192/192  4s 19ms/step - loss: 0.0042 - val_loss: 0.0043
Epoch 12/50
192/192  4s 19ms/step - loss: 0.0040 - val_loss: 0.0043
Epoch 13/50
192/192  4s 19ms/step - loss: 0.0039 - val_loss: 0.0041
Epoch 14/50
192/192  4s 19ms/step - loss: 0.0038 - val_loss: 0.0040
Epoch 15/50
192/192  4s 19ms/step - loss: 0.0037 - val_loss: 0.0039
Epoch 16/50
192/192  4s 19ms/step - loss: 0.0036 - val_loss: 0.0038
Epoch 17/50
192/192  4s 19ms/step - loss: 0.0035 - val_loss: 0.0041
Epoch 18/50
192/192  4s 19ms/step - loss: 0.0034 - val_loss: 0.0037
Epoch 19/50
192/192  4s 18ms/step - loss: 0.0034 - val_loss: 0.0037
Epoch 20/50
192/192  4s 20ms/step - loss: 0.0033 - val_loss: 0.0035
Epoch 21/50
192/192  4s 18ms/step - loss: 0.0032 - val_loss: 0.0036
Epoch 22/50
192/192  4s 18ms/step - loss: 0.0032 - val_loss: 0.0035
Epoch 23/50
192/192  4s 20ms/step - loss: 0.0032 - val_loss: 0.0035
Epoch 24/50
192/192  4s 19ms/step - loss: 0.0031 - val_loss: 0.0036
Epoch 25/50
192/192  4s 18ms/step - loss: 0.0031 - val_loss: 0.0034
Epoch 26/50
192/192  4s 18ms/step - loss: 0.0030 - val_loss: 0.0035
Epoch 27/50
192/192  4s 19ms/step - loss: 0.0030 - val_loss: 0.0033
Epoch 28/50
192/192  4s 18ms/step - loss: 0.0030 - val_loss: 0.0033
Epoch 29/50
192/192  4s 18ms/step - loss: 0.0029 - val_loss: 0.0034
Epoch 30/50
192/192  4s 19ms/step - loss: 0.0029 - val_loss: 0.0034
Epoch 31/50
192/192  4s 19ms/step - loss: 0.0029 - val_loss: 0.0032
Epoch 32/50
192/192  4s 19ms/step - loss: 0.0028 - val_loss: 0.0032
Epoch 33/50
192/192  4s 18ms/step - loss: 0.0028 - val_loss: 0.0032
Epoch 34/50

192/192  **4s** 19ms/step - loss: 0.0028 - val_loss: 0.0032
 Epoch 35/50
192/192  **4s** 18ms/step - loss: 0.0028 - val_loss: 0.0032
 Epoch 36/50
192/192  **4s** 19ms/step - loss: 0.0027 - val_loss: 0.0032
 Epoch 37/50
192/192  **4s** 19ms/step - loss: 0.0027 - val_loss: 0.0032
 Epoch 38/50
192/192  **4s** 19ms/step - loss: 0.0027 - val_loss: 0.0032
 Epoch 39/50
192/192  **4s** 18ms/step - loss: 0.0027 - val_loss: 0.0031
 Epoch 40/50
192/192  **4s** 19ms/step - loss: 0.0027 - val_loss: 0.0032
 Epoch 41/50
192/192  **4s** 20ms/step - loss: 0.0027 - val_loss: 0.0031
 Epoch 42/50
192/192  **4s** 18ms/step - loss: 0.0026 - val_loss: 0.0033
 Epoch 43/50
192/192  **4s** 19ms/step - loss: 0.0026 - val_loss: 0.0031
 Epoch 44/50
192/192  **4s** 20ms/step - loss: 0.0026 - val_loss: 0.0032
 Epoch 45/50
192/192  **4s** 18ms/step - loss: 0.0026 - val_loss: 0.0032
 Epoch 46/50
192/192  **4s** 18ms/step - loss: 0.0026 - val_loss: 0.0031
 Epoch 47/50
192/192  **4s** 19ms/step - loss: 0.0026 - val_loss: 0.0031
 Epoch 48/50
192/192  **4s** 19ms/step - loss: 0.0026 - val_loss: 0.0031
 Epoch 49/50
192/192  **4s** 19ms/step - loss: 0.0025 - val_loss: 0.0030
 Epoch 50/50
192/192  **4s** 19ms/step - loss: 0.0025 - val_loss: 0.0030

Task 3 models for CIFAR10-gray

Epoch 1/60
165/165  **3s** 10ms/step - loss: 0.0654 - val_loss: 0.0410
 Epoch 2/60
165/165  **1s** 4ms/step - loss: 0.0366 - val_loss: 0.0329
 Epoch 3/60
165/165  **1s** 4ms/step - loss: 0.0306 - val_loss: 0.0283
 Epoch 4/60
165/165  **1s** 4ms/step - loss: 0.0261 - val_loss: 0.0239
 Epoch 5/60
165/165  **1s** 4ms/step - loss: 0.0217 - val_loss: 0.0204
 Epoch 6/60
165/165  **1s** 4ms/step - loss: 0.0184 - val_loss: 0.0172
 Epoch 7/60
165/165  **1s** 4ms/step - loss: 0.0161 - val_loss: 0.0152
 Epoch 8/60
165/165  **1s** 4ms/step - loss: 0.0145 - val_loss: 0.0140
 Epoch 9/60
165/165  **1s** 4ms/step - loss: 0.0135 - val_loss: 0.0131
 Epoch 10/60
165/165  **1s** 4ms/step - loss: 0.0126 - val_loss: 0.0123
 Epoch 11/60
165/165  **1s** 5ms/step - loss: 0.0119 - val_loss: 0.0120
 Epoch 12/60
165/165  **1s** 6ms/step - loss: 0.0113 - val_loss: 0.0112
 Epoch 13/60
165/165  **1s** 6ms/step - loss: 0.0108 - val_loss: 0.0106
 Epoch 14/60
165/165  **1s** 4ms/step - loss: 0.0104 - val_loss: 0.0103
 Epoch 15/60
165/165  **1s** 5ms/step - loss: 0.0101 - val_loss: 0.0101
 Epoch 16/60
165/165  **1s** 4ms/step - loss: 0.0099 - val_loss: 0.0098

Epoch 17/60
165/165  1s 4ms/step - loss: 0.0096 - val_loss: 0.0098
Epoch 18/60
165/165  1s 4ms/step - loss: 0.0095 - val_loss: 0.0094
Epoch 19/60
165/165  1s 4ms/step - loss: 0.0093 - val_loss: 0.0095
Epoch 20/60
165/165  1s 4ms/step - loss: 0.0092 - val_loss: 0.0092
Epoch 21/60
165/165  1s 4ms/step - loss: 0.0091 - val_loss: 0.0091
Epoch 22/60
165/165  1s 3ms/step - loss: 0.0090 - val_loss: 0.0091
Epoch 23/60
165/165  1s 4ms/step - loss: 0.0090 - val_loss: 0.0091
Epoch 24/60
165/165  1s 4ms/step - loss: 0.0089 - val_loss: 0.0090
Epoch 25/60
165/165  1s 4ms/step - loss: 0.0089 - val_loss: 0.0089
Epoch 26/60
165/165  1s 4ms/step - loss: 0.0088 - val_loss: 0.0089
Epoch 27/60
165/165  1s 4ms/step - loss: 0.0088 - val_loss: 0.0090
Epoch 28/60
165/165  1s 4ms/step - loss: 0.0088 - val_loss: 0.0090
Epoch 29/60
165/165  1s 6ms/step - loss: 0.0088 - val_loss: 0.0090
Epoch 30/60
165/165  1s 6ms/step - loss: 0.0088 - val_loss: 0.0091
Epoch 31/60
165/165  1s 4ms/step - loss: 0.0087 - val_loss: 0.0088
Epoch 32/60
165/165  1s 4ms/step - loss: 0.0087 - val_loss: 0.0088
Epoch 33/60
165/165  1s 4ms/step - loss: 0.0087 - val_loss: 0.0087
Epoch 34/60
165/165  1s 4ms/step - loss: 0.0087 - val_loss: 0.0087
Epoch 35/60
165/165  1s 4ms/step - loss: 0.0087 - val_loss: 0.0087
Epoch 36/60
165/165  1s 4ms/step - loss: 0.0086 - val_loss: 0.0087
Epoch 37/60
165/165  1s 4ms/step - loss: 0.0086 - val_loss: 0.0095
Epoch 38/60
165/165  1s 3ms/step - loss: 0.0086 - val_loss: 0.0087
Epoch 39/60
165/165  1s 4ms/step - loss: 0.0086 - val_loss: 0.0086
Epoch 40/60
165/165  1s 4ms/step - loss: 0.0086 - val_loss: 0.0087
Epoch 41/60
165/165  1s 4ms/step - loss: 0.0086 - val_loss: 0.0086
Epoch 42/60
165/165  1s 4ms/step - loss: 0.0085 - val_loss: 0.0086
Epoch 43/60
165/165  1s 4ms/step - loss: 0.0085 - val_loss: 0.0086
Epoch 44/60
165/165  1s 5ms/step - loss: 0.0085 - val_loss: 0.0085
Epoch 45/60
165/165  1s 5ms/step - loss: 0.0085 - val_loss: 0.0086
Epoch 46/60
165/165  1s 6ms/step - loss: 0.0085 - val_loss: 0.0086
Epoch 47/60
165/165  1s 4ms/step - loss: 0.0085 - val_loss: 0.0088
Epoch 48/60
165/165  1s 4ms/step - loss: 0.0085 - val_loss: 0.0085
Epoch 49/60
165/165  1s 4ms/step - loss: 0.0084 - val_loss: 0.0085
Epoch 50/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0085
Epoch 51/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0085
Epoch 52/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0085
Epoch 53/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0085
Epoch 54/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0085
Epoch 55/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0085
Epoch 56/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0086
Epoch 57/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0085
Epoch 58/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0087
Epoch 59/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0084
Epoch 60/60

165/165 1s 4ms/step - loss: 0.0084 - val_loss: 0.0084
Epoch 1/60

165/165 4s 12ms/step - loss: 0.0962 - val_loss: 0.0530
Epoch 2/60

165/165 1s 4ms/step - loss: 0.0517 - val_loss: 0.0504
Epoch 3/60

165/165 1s 4ms/step - loss: 0.0495 - val_loss: 0.0484
Epoch 4/60

165/165 1s 4ms/step - loss: 0.0474 - val_loss: 0.0455
Epoch 5/60

165/165 1s 4ms/step - loss: 0.0436 - val_loss: 0.0411
Epoch 6/60

165/165 1s 4ms/step - loss: 0.0391 - val_loss: 0.0371
Epoch 7/60

165/165 1s 4ms/step - loss: 0.0359 - val_loss: 0.0348
Epoch 8/60

165/165 1s 4ms/step - loss: 0.0338 - val_loss: 0.0328
Epoch 9/60

165/165 1s 4ms/step - loss: 0.0321 - val_loss: 0.0315
Epoch 10/60

165/165 1s 4ms/step - loss: 0.0311 - val_loss: 0.0309
Epoch 11/60

165/165 1s 5ms/step - loss: 0.0305 - val_loss: 0.0304
Epoch 12/60

165/165 1s 7ms/step - loss: 0.0302 - val_loss: 0.0301
Epoch 13/60

165/165 1s 5ms/step - loss: 0.0300 - val_loss: 0.0300
Epoch 14/60

165/165 1s 4ms/step - loss: 0.0299 - val_loss: 0.0300
Epoch 15/60

165/165 1s 4ms/step - loss: 0.0298 - val_loss: 0.0297
Epoch 16/60

165/165 1s 4ms/step - loss: 0.0296 - val_loss: 0.0297
Epoch 17/60

165/165 1s 4ms/step - loss: 0.0294 - val_loss: 0.0293
Epoch 18/60

165/165 1s 4ms/step - loss: 0.0289 - val_loss: 0.0284
Epoch 19/60

165/165 1s 4ms/step - loss: 0.0276 - val_loss: 0.0270
Epoch 20/60

165/165 1s 4ms/step - loss: 0.0265 - val_loss: 0.0263
Epoch 21/60

165/165 1s 4ms/step - loss: 0.0261 - val_loss: 0.0261
Epoch 22/60

165/165 1s 4ms/step - loss: 0.0259 - val_loss: 0.0259
Epoch 23/60

165/165 1s 4ms/step - loss: 0.0258 - val_loss: 0.0258

Epoch 24/60
165/165  1s 4ms/step - loss: 0.0257 - val_loss: 0.0257
Epoch 25/60
165/165  1s 4ms/step - loss: 0.0256 - val_loss: 0.0257
Epoch 26/60
165/165  1s 4ms/step - loss: 0.0256 - val_loss: 0.0256
Epoch 27/60
165/165  1s 4ms/step - loss: 0.0255 - val_loss: 0.0256
Epoch 28/60
165/165  1s 6ms/step - loss: 0.0255 - val_loss: 0.0256
Epoch 29/60
165/165  1s 6ms/step - loss: 0.0255 - val_loss: 0.0256
Epoch 30/60
165/165  1s 5ms/step - loss: 0.0255 - val_loss: 0.0255
Epoch 31/60
165/165  1s 4ms/step - loss: 0.0255 - val_loss: 0.0255
Epoch 32/60
165/165  1s 4ms/step - loss: 0.0254 - val_loss: 0.0255
Epoch 33/60
165/165  1s 4ms/step - loss: 0.0254 - val_loss: 0.0255
Epoch 34/60
165/165  1s 4ms/step - loss: 0.0254 - val_loss: 0.0254
Epoch 35/60
165/165  1s 4ms/step - loss: 0.0254 - val_loss: 0.0254
Epoch 36/60
165/165  1s 4ms/step - loss: 0.0253 - val_loss: 0.0253
Epoch 37/60
165/165  1s 4ms/step - loss: 0.0252 - val_loss: 0.0253
Epoch 38/60
165/165  1s 4ms/step - loss: 0.0250 - val_loss: 0.0252
Epoch 39/60
165/165  1s 4ms/step - loss: 0.0248 - val_loss: 0.0246
Epoch 40/60
165/165  1s 4ms/step - loss: 0.0243 - val_loss: 0.0245
Epoch 41/60
165/165  1s 4ms/step - loss: 0.0240 - val_loss: 0.0240
Epoch 42/60
165/165  1s 4ms/step - loss: 0.0239 - val_loss: 0.0241
Epoch 43/60
165/165  1s 4ms/step - loss: 0.0238 - val_loss: 0.0238
Epoch 44/60
165/165  1s 7ms/step - loss: 0.0237 - val_loss: 0.0237
Epoch 45/60
165/165  1s 7ms/step - loss: 0.0236 - val_loss: 0.0237
Epoch 46/60
165/165  1s 4ms/step - loss: 0.0236 - val_loss: 0.0236
Epoch 47/60
165/165  1s 4ms/step - loss: 0.0235 - val_loss: 0.0237
Epoch 48/60
165/165  1s 4ms/step - loss: 0.0235 - val_loss: 0.0236
Epoch 49/60
165/165  1s 4ms/step - loss: 0.0235 - val_loss: 0.0235
Epoch 50/60
165/165  1s 4ms/step - loss: 0.0234 - val_loss: 0.0235
Epoch 51/60
165/165  1s 4ms/step - loss: 0.0234 - val_loss: 0.0234
Epoch 52/60
165/165  1s 4ms/step - loss: 0.0234 - val_loss: 0.0234
Epoch 53/60
165/165  1s 4ms/step - loss: 0.0234 - val_loss: 0.0234
Epoch 54/60
165/165  1s 4ms/step - loss: 0.0233 - val_loss: 0.0235
Epoch 55/60
165/165  1s 4ms/step - loss: 0.0233 - val_loss: 0.0234
Epoch 56/60
165/165  1s 4ms/step - loss: 0.0233 - val_loss: 0.0233
Epoch 57/60

165/165 ————— 1s 4ms/step - loss: 0.0233 - val_loss: 0.0234
Epoch 58/60
165/165 ————— 1s 5ms/step - loss: 0.0233 - val_loss: 0.0234
Epoch 59/60
165/165 ————— 1s 6ms/step - loss: 0.0232 - val_loss: 0.0234
Epoch 60/60
165/165 ————— 1s 7ms/step - loss: 0.0232 - val_loss: 0.0235
Model: "deep_convolutional_ae_latent30"

Layer (type)	Output Shape	Param #
image_28x28x1 (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	73,856
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 128)	802,944
latent_30 (Dense)	(None, 30)	3,870
dense_1 (Dense)	(None, 6272)	194,432
reshape (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	73,792
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	18,464
linear_decoder (Conv2D)	(None, 28, 28, 1)	289

Total params: 1,186,463 (4.53 MB)
Trainable params: 1,186,463 (4.53 MB)
Non-trainable params: 0 (0.00 B)

Epoch 1/50
165/165  **13s** 51ms/step - loss: 0.0496 - val_loss: 0.0226

Epoch 2/50
165/165  **3s** 19ms/step - loss: 0.0164 - val_loss: 0.0132

Epoch 3/50
165/165  **3s** 19ms/step - loss: 0.0121 - val_loss: 0.0115

Epoch 4/50
165/165  **3s** 20ms/step - loss: 0.0107 - val_loss: 0.0102

Epoch 5/50
165/165  **3s** 20ms/step - loss: 0.0098 - val_loss: 0.0097

Epoch 6/50
165/165  **3s** 19ms/step - loss: 0.0091 - val_loss: 0.0097

Epoch 7/50
165/165  **3s** 19ms/step - loss: 0.0087 - val_loss: 0.0085

Epoch 8/50
165/165  **3s** 20ms/step - loss: 0.0084 - val_loss: 0.0083

Epoch 9/50
165/165  **3s** 20ms/step - loss: 0.0082 - val_loss: 0.0082

Epoch 10/50
165/165  **3s** 19ms/step - loss: 0.0081 - val_loss: 0.0081

Epoch 11/50
165/165  **3s** 19ms/step - loss: 0.0079 - val_loss: 0.0087

Epoch 12/50
165/165  **3s** 20ms/step - loss: 0.0078 - val_loss: 0.0078

Epoch 13/50
165/165  **3s** 19ms/step - loss: 0.0078 - val_loss: 0.0078

Epoch 14/50
165/165  **3s** 19ms/step - loss: 0.0077 - val_loss: 0.0078

Epoch 15/50
165/165  **3s** 19ms/step - loss: 0.0077 - val_loss: 0.0078

Epoch 16/50
165/165  **4s** 21ms/step - loss: 0.0077 - val_loss: 0.0077

Epoch 17/50
165/165  **3s** 19ms/step - loss: 0.0076 - val_loss: 0.0078

Epoch 18/50
165/165  **5s** 19ms/step - loss: 0.0076 - val_loss: 0.0077

Epoch 19/50
165/165  **3s** 20ms/step - loss: 0.0076 - val_loss: 0.0080

Epoch 20/50
165/165  **5s** 19ms/step - loss: 0.0075 - val_loss: 0.0077

Epoch 21/50
165/165  **3s** 19ms/step - loss: 0.0075 - val_loss: 0.0077

Epoch 22/50
165/165  **3s** 19ms/step - loss: 0.0074 - val_loss: 0.0077

Epoch 23/50
165/165  **3s** 20ms/step - loss: 0.0074 - val_loss: 0.0077

Epoch 24/50
165/165  **3s** 19ms/step - loss: 0.0074 - val_loss: 0.0076

Epoch 25/50
165/165  **3s** 19ms/step - loss: 0.0074 - val_loss: 0.0078

Epoch 26/50
165/165  **3s** 19ms/step - loss: 0.0074 - val_loss: 0.0076

Epoch 27/50
165/165  **3s** 20ms/step - loss: 0.0073 - val_loss: 0.0079

Epoch 28/50
165/165  **3s** 19ms/step - loss: 0.0073 - val_loss: 0.0076

Epoch 29/50
165/165  **3s** 19ms/step - loss: 0.0073 - val_loss: 0.0076

Epoch 30/50
165/165  **3s** 20ms/step - loss: 0.0072 - val_loss: 0.0076

Epoch 31/50
165/165  **3s** 19ms/step - loss: 0.0073 - val_loss: 0.0076

Epoch 32/50
165/165  **3s** 19ms/step - loss: 0.0072 - val_loss: 0.0075

Epoch 33/50
165/165  **3s** 19ms/step - loss: 0.0073 - val_loss: 0.0076

Epoch 34/50

```

165/165 ————— 3s 20ms/step - loss: 0.0072 - val_loss: 0.0077
Epoch 35/50
165/165 ————— 3s 19ms/step - loss: 0.0072 - val_loss: 0.0075
Epoch 36/50
165/165 ————— 3s 19ms/step - loss: 0.0071 - val_loss: 0.0075
Epoch 37/50
165/165 ————— 3s 19ms/step - loss: 0.0071 - val_loss: 0.0081
Epoch 38/50
165/165 ————— 4s 21ms/step - loss: 0.0072 - val_loss: 0.0075
Epoch 39/50
165/165 ————— 3s 19ms/step - loss: 0.0071 - val_loss: 0.0077
Epoch 40/50
165/165 ————— 3s 19ms/step - loss: 0.0071 - val_loss: 0.0076
Epoch 41/50
165/165 ————— 3s 19ms/step - loss: 0.0071 - val_loss: 0.0075
Epoch 42/50
165/165 ————— 3s 20ms/step - loss: 0.0071 - val_loss: 0.0075
Epoch 43/50
165/165 ————— 3s 19ms/step - loss: 0.0070 - val_loss: 0.0075
Epoch 44/50
165/165 ————— 5s 19ms/step - loss: 0.0070 - val_loss: 0.0076
Epoch 45/50
165/165 ————— 3s 19ms/step - loss: 0.0071 - val_loss: 0.0075
Epoch 46/50
165/165 ————— 3s 19ms/step - loss: 0.0070 - val_loss: 0.0081
Epoch 47/50
165/165 ————— 3s 19ms/step - loss: 0.0070 - val_loss: 0.0075
Epoch 48/50
165/165 ————— 3s 19ms/step - loss: 0.0070 - val_loss: 0.0076
Epoch 49/50
165/165 ————— 3s 19ms/step - loss: 0.0070 - val_loss: 0.0076
Epoch 50/50
165/165 ————— 3s 19ms/step - loss: 0.0069 - val_loss: 0.0075

```

Out[16]:

	dataset	method	average_test_snr_db
0	MNIST	single hidden dense AE, sigmoid encoder 30, li...	12.950556
1	MNIST	three hidden dense AE, sigmoid 10-10-10, linea...	9.667780
2	MNIST	deep convolutional AE, latent=30	20.967989
3	CIFAR10-gray	single hidden dense AE, sigmoid encoder 30, li...	19.835799
4	CIFAR10-gray	three hidden dense AE, sigmoid 10-10-10, linea...	15.498728
5	CIFAR10-gray	deep convolutional AE, latent=30	20.324699

```

In [17]: # Direct answer: average SNR when 30 nodes split equally as 10-10-10 across 3
hidden layers
print('Task 3 answer: average test SNR with 10-10-10 hidden layer distribution
(sigmoid hidden, linear decoder):')
print()
for _, row in task3_results[task3_results['method'].str.contains('three
hidden')].iterrows():
    print(f' {row["dataset"]}: {row["average_test_snr_db"]:.2f} dB')

```

Task 3 answer: average test SNR with 10-10-10 hidden layer distribution (sigmoid hid
den, linear decoder):

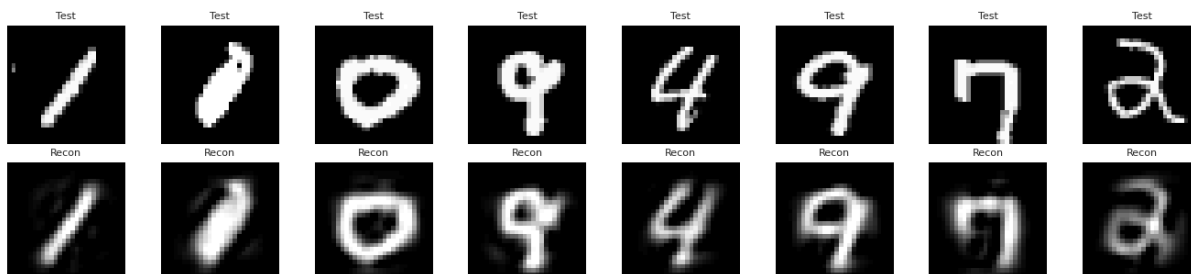
```

MNIST: 9.67 dB
CIFAR10-gray: 15.50 dB

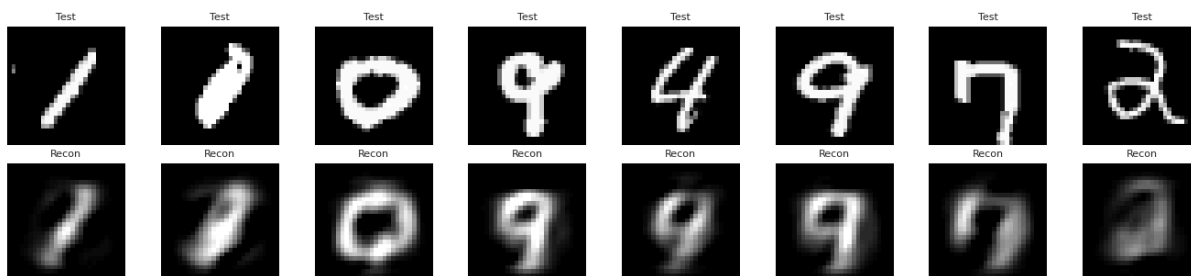
```

```
In [18]: for ds in datasets:
        for key, title in [
            ("single", "single hidden dense AE"),
            ("three", "three hidden dense AE, 10-10-10"),
            ("conv", "deep convolutional AE, latent=30"),
        ]:
            recon = task3_models[(ds.name, key)][f"recon_50_200"]
            plot_reconstruction_examples(ds.x_test_flat_50_200, recon, f"{ds.name}:
{title} reconstructions")
```

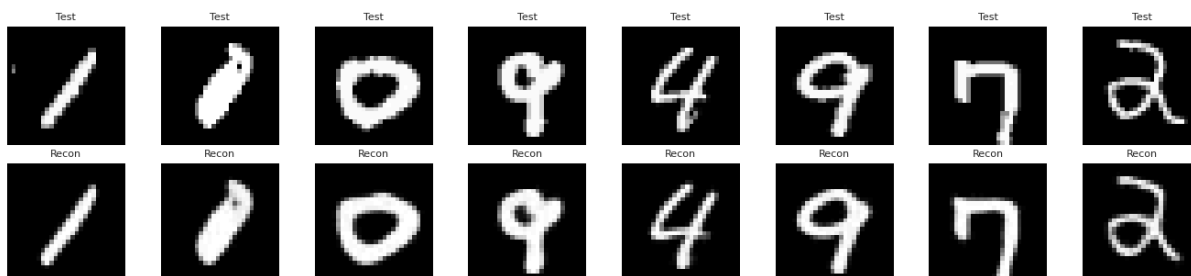
MNIST: single hidden dense AE reconstructions



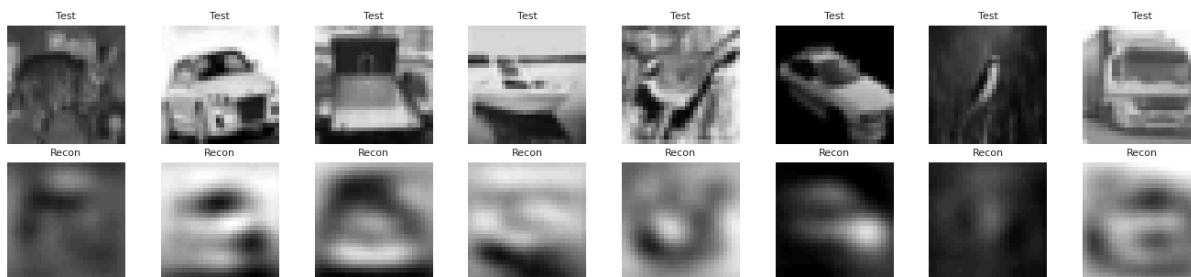
MNIST: three hidden dense AE, 10-10-10 reconstructions



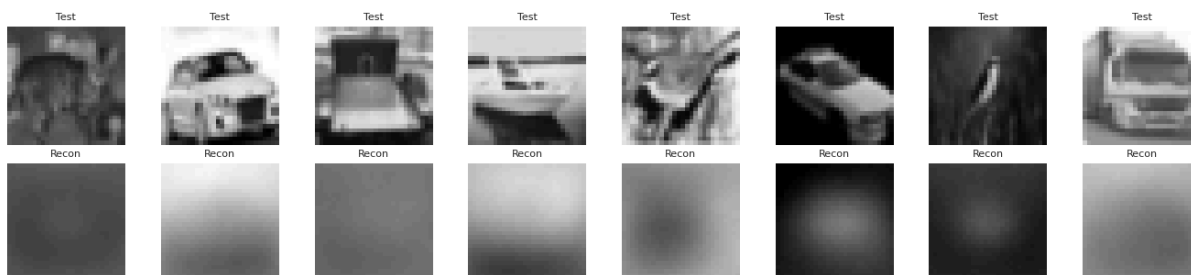
MNIST: deep convolutional AE, latent=30 reconstructions



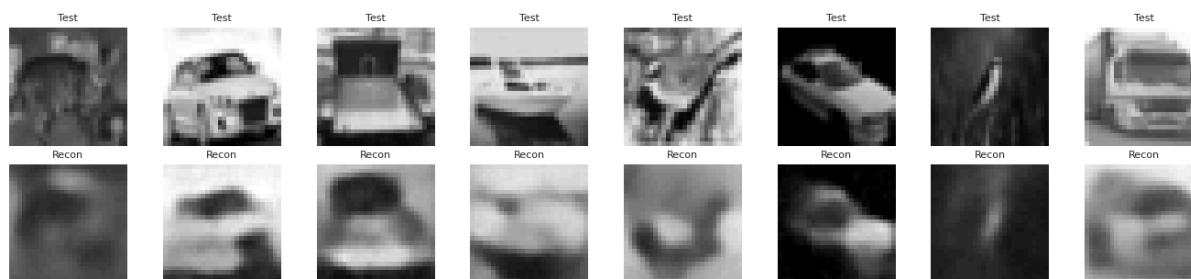
CIFAR10-gray: single hidden dense AE reconstructions



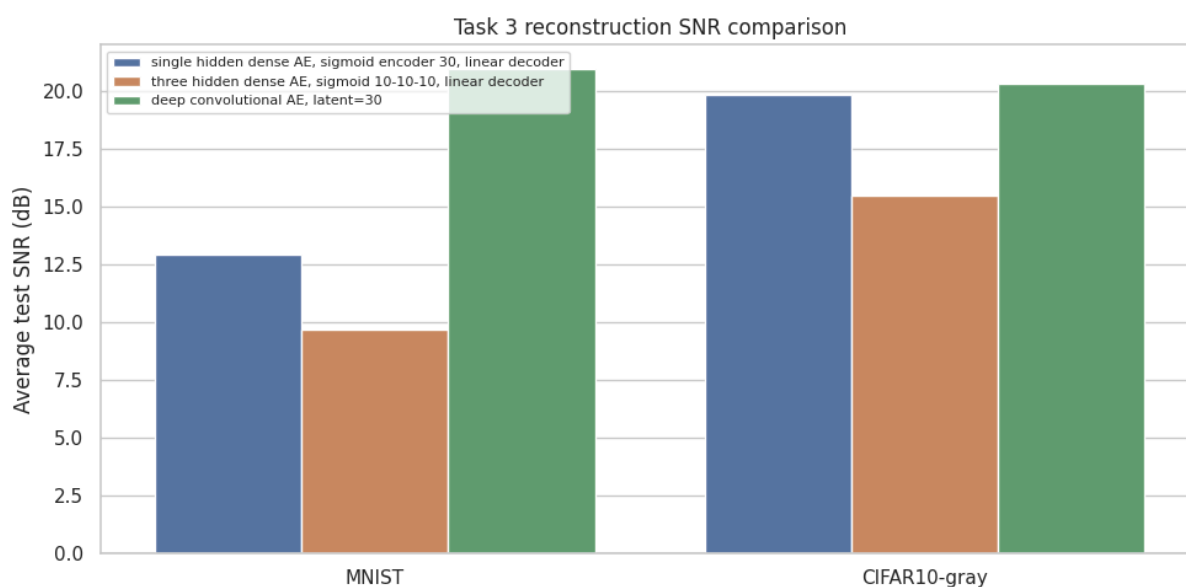
CIFAR10-gray: three hidden dense AE, 10-10-10 reconstructions



CIFAR10-gray: deep convolutional AE, latent=30 reconstructions



```
In [19]: plt.figure(figsize=(10, 5))
sns.barplot(data=task3_results, x="dataset", y="average_test_snr_db", hue="method")
plt.title("Task 3 reconstruction SNR comparison")
plt.ylabel("Average test SNR (dB)")
plt.xlabel("")
plt.xticks(rotation=0)
plt.legend(loc="best", fontsize=8)
plt.tight_layout()
plt.show()
```



Task 3 comments.

The convolutional autoencoder is expected to achieve the highest SNR because convolutional filters exploit spatial locality and weight sharing. The single hidden layer dense autoencoder with only 30 sigmoid hidden nodes is a much stronger bottleneck and ignores image geometry. The three-hidden-layer dense model uses the same total hidden-node budget split as 10-10-10, so the effective bottleneck is only 10 dimensions; it normally reconstructs worse than the 30-node single hidden layer model, especially for gray CIFAR10. MNIST is usually easier to reconstruct than CIFAR10 because digits are sparse and structurally simpler.

Final Summary Tables

```
In [20]: print("Task 1: PCA and randomized PCA")
display(task1_results)

print("Task 1: top 30 eigenvalues")
display(task1_top30_eigenvalues)

print("Task 2: PCA versus tied linear AE classification and SNR")
display(
    pd.concat(
        [
            task1_results.query("method == 'standard PCA'")[
                ["dataset", "method", "components", "accuracy", "macro_auc_ovr",
                "average_test_snr_db"]
            ],
            task2_results,
        ],
        ignore_index=True,
    )
)

print("Task 2: quantitative component relationship")
display(component_comparison)

print("Task 3: nonlinear autoencoder SNR")
display(task3_results)
```

Task 1: PCA and randomized PCA

	dataset	method	components	accuracy	macro_auc_ovr	explained_variance_ratio_sum	average_test_snr_db
0	MNIST	standard PCA	30	0.893571	0.990232	0.731537	12.572198
1	MNIST	randomized PCA	30	0.893429	0.990141	0.731483	12.571397
2	CIFAR10-gray	standard PCA	30	0.307000	0.729698	0.849658	20.001223
3	CIFAR10-gray	randomized PCA	30	0.307167	0.729697	0.849626	20.001208

Task 1: top 30 eigenvalues

	dataset	method	principal_component	eigenvalue	explained_variance_ratio
0	MNIST	standard PCA	1	115334.734375	0.097198
1	MNIST	standard PCA	2	84966.359375	0.071605
2	MNIST	standard PCA	3	72920.585938	0.061453
3	MNIST	standard PCA	4	64178.867188	0.054086
4	MNIST	standard PCA	5	57888.882812	0.048785
...	...	...	...	...	...
115	CIFAR10-gray	randomized PCA	26	4224.890625	0.004475
116	CIFAR10-gray	randomized PCA	27	3719.742920	0.003940
117	CIFAR10-gray	randomized PCA	28	3548.849121	0.003759
118	CIFAR10-gray	randomized PCA	29	3444.521240	0.003648
119	CIFAR10-gray	randomized PCA	30	3264.271973	0.003457

120 rows × 5 columns

Task 2: PCA versus tied linear AE classification and SNR

	dataset	method	components	accuracy	macro_auc_ovr	average_test_snr_db
0	MNIST	standard PCA	30	0.893571	0.990232	12.572198
1	CIFAR10-gray	standard PCA	30	0.307000	0.729698	20.001223
2	MNIST	tied linear AE features	30	0.891000	0.989863	11.217112
3	CIFAR10-gray	tied linear AE features	30	0.305500	0.729356	19.984097

Task 2: quantitative component relationship

	dataset	mean_matched_abs_cosine	median_matched_abs_cosine	min_matched_abs_cosine	mean_subspace_singular_value	max_principal_angle_deg
0	MNIST	0.005936	0.004382	0.001041	0.013963	89.981156
1	CIFAR10-gray	0.395424	0.391600	0.203096	0.986817	32.385975

Task 3: nonlinear autoencoder SNR

	dataset	method	average_test_snr_db
0	MNIST	single hidden dense AE, sigmoid encoder 30, li...	12.950556
1	MNIST	three hidden dense AE, sigmoid 10-10-10, linea...	9.667780
2	MNIST	deep convolutional AE, latent=30	20.967989
3	CIFAR10-gray	single hidden dense AE, sigmoid encoder 30, li...	19.835799
4	CIFAR10-gray	three hidden dense AE, sigmoid 10-10-10, linea...	15.498728
5	CIFAR10-gray	deep convolutional AE, latent=30	20.324699

```
In [21]: def print_final_observations():
    print("Final observations generated from the current run")
    for ds_name in task1_results["dataset"].unique():
        std = task1_results[(task1_results["dataset"] == ds_name) &
(task1_results["method"] == "standard PCA")].iloc[0]
        rnd = task1_results[(task1_results["dataset"] == ds_name) &
(task1_results["method"] == "randomized PCA")].iloc[0]
        ae = task2_results[task2_results["dataset"] == ds_name].iloc[0]
        comp = component_comparison[component_comparison["dataset"] ==
ds_name].iloc[0]
        single = task3_results[(task3_results["dataset"] == ds_name) &
task3_results["method"].str.contains("single hidden", regex=False)].iloc[0]
        three = task3_results[(task3_results["dataset"] == ds_name) &
task3_results["method"].str.contains("three hidden", regex=False)].iloc[0]
        conv = task3_results[(task3_results["dataset"] == ds_name) &
task3_results["method"].str.contains("convolutional", regex=False)].iloc[0]

        print(f"\n{ds_name}")
        print(f"    Task 1 PCA          : acc={std['accuracy']:.4f}, macro AUC=
{std['macro_auc_ovr']:.4f}, SNR={std['average_test_snr_db']:.2f} dB")
        print(f"    Task 1 randomized: acc={rnd['accuracy']:.4f}, macro AUC=
{rnd['macro_auc_ovr']:.4f}, SNR={rnd['average_test_snr_db']:.2f} dB")
        print(f"    Task 2 tied AE    : acc={ae['accuracy']:.4f}, macro AUC=
{ae['macro_auc_ovr']:.4f}, SNR={ae['average_test_snr_db']:.2f} dB")
        print(f"    PCA-AE relation : mean matched |cos|=
{comp['mean_matched_abs_cosine']:.4f}, max principal angle=
{comp['max_principal_angle_deg']:.2f} degrees")
        print(f"    Task 3 single AE : SNR={single['average_test_snr_db']:.2f} dB")
        print(f"    Task 3 10-10-10 : SNR={three['average_test_snr_db']:.2f} dB")
        print(f"    Task 3 conv AE   : SNR={conv['average_test_snr_db']:.2f} dB")

    print("\nConclusion: randomized PCA should closely match standard PCA; the tied
linear AE should recover a PCA-like subspace; "
        "and the convolutional AE is expected to reconstruct best because it uses
spatial structure.")

print_final_observations()
```

Final observations generated from the current run

MNIST

```
Task 1 PCA          : acc=0.8936, macro AUC=0.9902, SNR=12.57 dB
Task 1 randomized: acc=0.8934, macro AUC=0.9901, SNR=12.57 dB
Task 2 tied AE     : acc=0.8910, macro AUC=0.9899, SNR=11.22 dB
PCA-AE relation    : mean matched |cos|=0.0059, max principal angle=89.98 degrees
Task 3 single AE   : SNR=12.95 dB
Task 3 10-10-10    : SNR=9.67 dB
Task 3 conv AE     : SNR=20.97 dB
```

CIFAR10-gray

```
Task 1 PCA          : acc=0.3070, macro AUC=0.7297, SNR=20.00 dB
Task 1 randomized: acc=0.3072, macro AUC=0.7297, SNR=20.00 dB
Task 2 tied AE     : acc=0.3055, macro AUC=0.7294, SNR=19.98 dB
PCA-AE relation    : mean matched |cos|=0.3954, max principal angle=32.39 degrees
Task 3 single AE   : SNR=19.84 dB
Task 3 10-10-10    : SNR=15.50 dB
Task 3 conv AE     : SNR=20.32 dB
```

Conclusion: randomized PCA should closely match standard PCA; the tied linear AE should recover a PCA-like subspace; and the convolutional AE is expected to reconstruct best because it uses spatial structure.